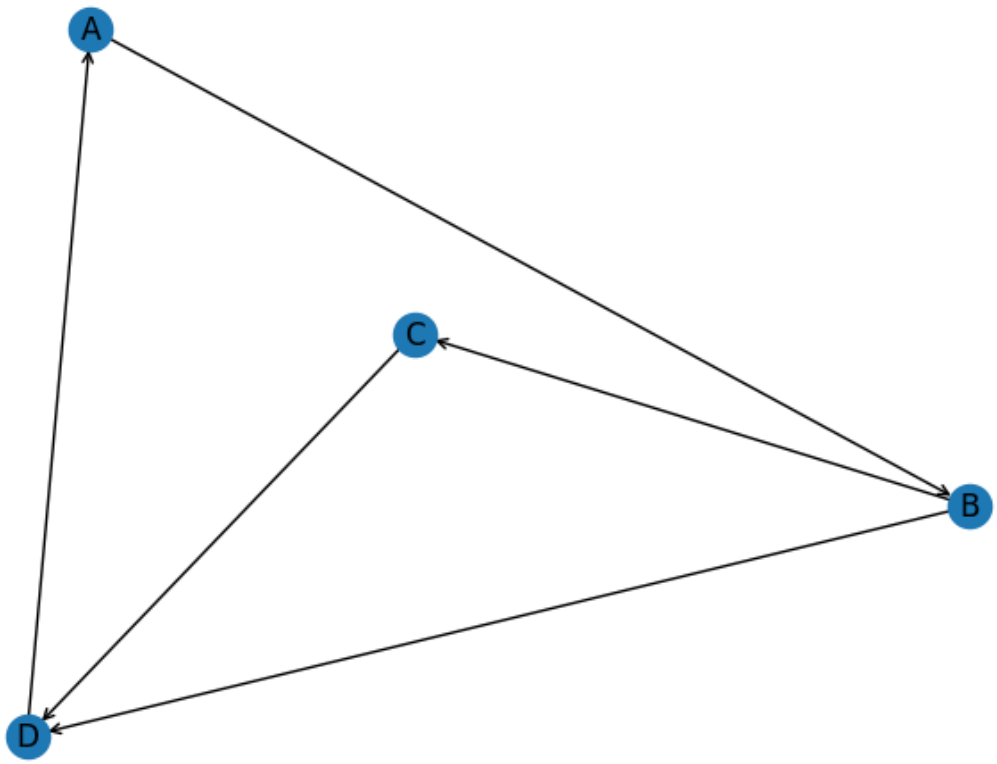
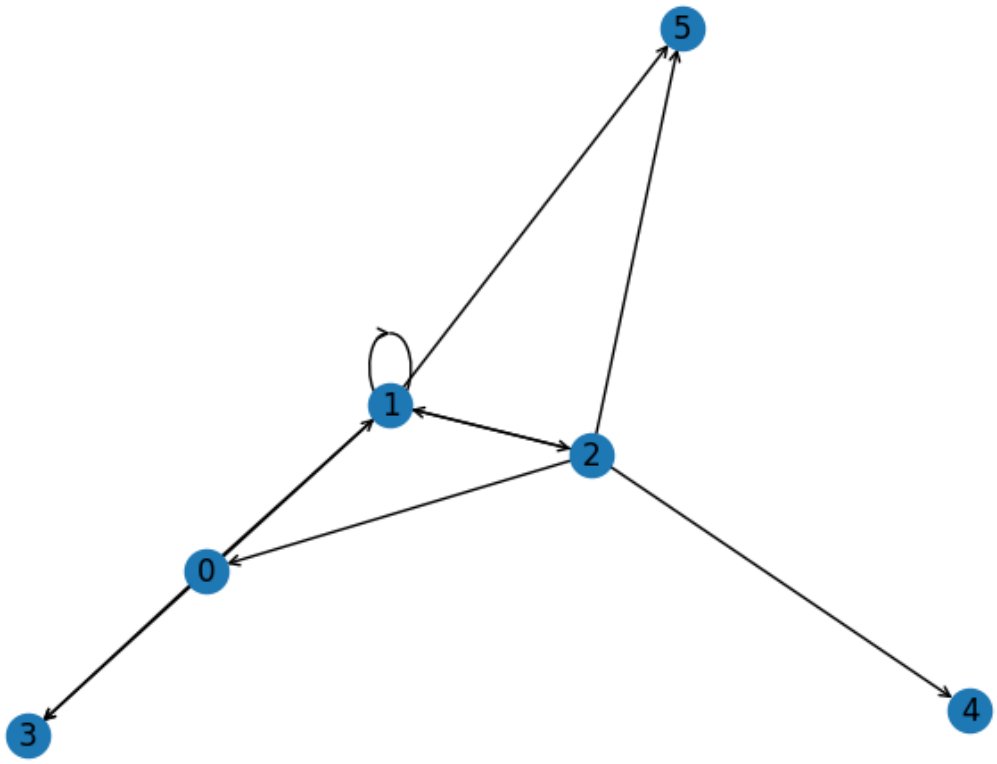
In [99]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
import plotly.express as px
import plotly.io as pio
```

In [100…
```
# 137

G=nx.DiGraph()
G.add_node('A')
G.add_node('B')
G.add_node('C')
G.add_node('D')
G.add_edges_from([
    ("A", "B"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),
    ("D", "A")
])

nx.draw(G,with_labels=True,arrows=True,arrowstyle="->")
```



In [101…
```
# 138

G=nx.DiGraph()
G.add_node('0')
G.add_node('1')
G.add_node('2')
```

```
G.add_node('3')
G.add_node('4')
G.add_node('5')
G.add_edges_from([
    ("0", "1"),
    ("1", "2"),
    ("1", "1"),
    ("2", "0"),
    ("2", "1"),
    ("1", "5"),
    ("2", "5"),
     ("2", "4"),
      ("0", "3"),
       ("1", "3")
])

nx.draw(G,with_labels=True,arrows=True,arrowstyle="->")
```



In [102…

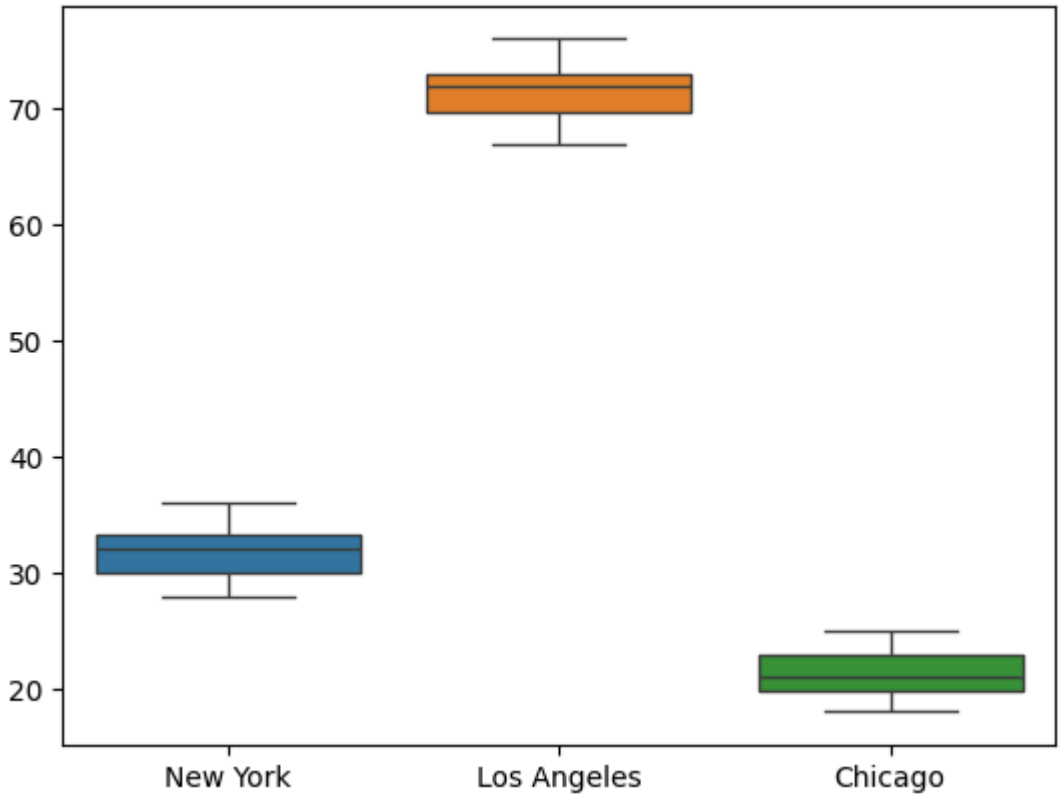
```
# 139

df = pd.read_csv("https://raw.githubusercontent.com/kavit88/Data-Sets/main/tempe
df
```

Out[102…

	New York	Los Angeles	Chicago	Date
0	30	70	20	01-05-2023
1	35	72	22	02-05-2023
2	28	68	18	03-05-2023
3	32	75	24	04-05-2023
4	33	73	21	05-05-2023
5	29	71	19	06-05-2023
6	31	74	23	07-05-2023
7	34	72	20	08-05-2023
8	30	70	22	09-05-2023
9	33	69	25	10-05-2023
10	32	76	20	11-05-2023
11	36	72	18	12-05-2023
12	31	68	23	13-05-2023
13	29	73	19	14-05-2023
14	32	75	24	15-05-2023
15	34	69	21	16-05-2023
16	30	71	18	17-05-2023
17	35	73	22	18-05-2023
18	28	67	20	19-05-2023
19	32	72	23	20-05-2023

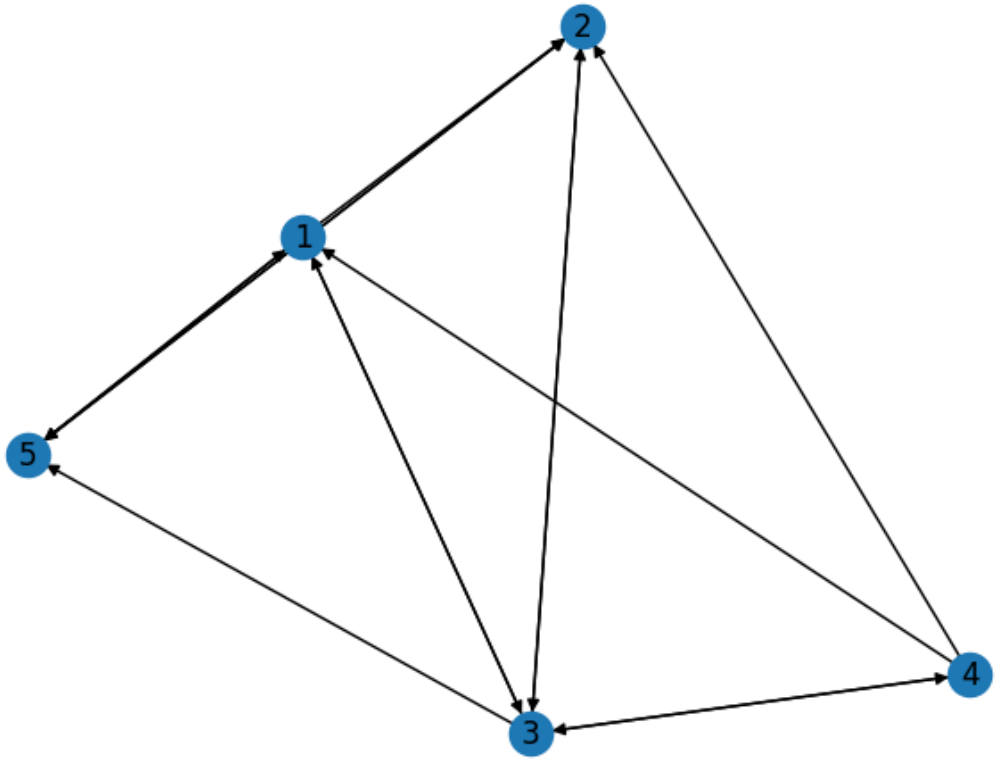
In [103…

```
sns.boxplot(data=df[["New York", "Los Angeles", "Chicago"]])
plt.show()
```



In [104…

```
# 140
G = nx.DiGraph()
instagram = {'person1': [0,1,1,0,1], 'person2': [0,0,1,0,1], 'person3': [1,1,0,1
G.add_edges_from([(1,2),(1,3),(1,5),(2,3),(2,5),(3,1),(3,2),(3,4),(3,5),(4,1),(4
nx.draw(G, with_labels=True)
plt.show()
```



In [105…

```
# 140

df=pd.read_csv("https://raw.githubusercontent.com/kavit88/Data-Sets/main/car_dat
df
```

Out[105…

	Price	Horsepower	Mileage	Weight
0	10000	120	25	2500
1	12000	140	20	3000
2	8000	100	30	2000
3	15000	180	18	3500
4	9000	110	28	2400
5	13000	150	22	3200
6	11000	130	24	2800
7	17000	200	16	4000
8	7500	90	32	1800
9	14000	170	20	3300
10	9500	105	27	2200
11	10500	125	26	2600
12	12500	145	21	3100
13	15500	190	17	3800
14	8500	95	31	1900
15	11500	135	23	2700
16	16500	195	15	3900
17	7000	85	35	1700
18	14500	175	19	3400
19	9800	115	29	2300
20	13500	160	21	3000
21	11800	140	23	2700
22	12800	150	22	3100
23	8000	100	31	2000
24	9500	110	28	2400

In [106…

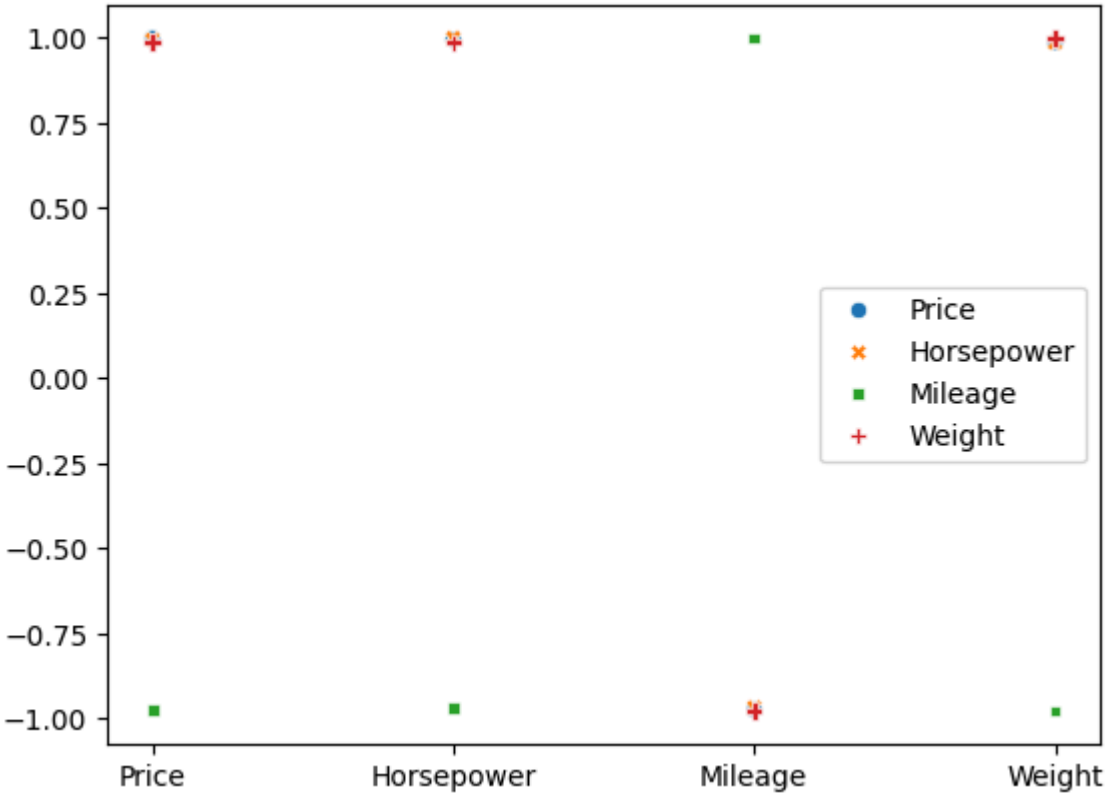
```
s=df.corr()
s
```

Out[106…

	Price	Horsepower	Mileage	Weight
Price	1.000000	0.995037	-0.973352	0.987126
Horsepower	0.995037	1.000000	-0.966742	0.986031
Mileage	-0.973352	-0.966742	1.000000	-0.976501
Weight	0.987126	0.986031	-0.976501	1.000000

In [107…

```
sns.scatterplot(s)
plt.show()
```



In [108…

```
# 141

df =pd.read_csv("https://raw.githubusercontent.com/kavit88/Data-Sets/main/house_
df
```

Out[108…

	Price	LotSize	SqFt
0	5000000	5000	2000
1	7500000	7500	2500
2	4500000	5500	1800
3	6000000	6500	2200
4	4000000	4500	1600
5	5500000	6000	2000
6	8500000	8000	3000
7	7000000	7000	2500
8	6500000	6500	2400
9	8000000	7500	2800
10	4500000	4500	1500
11	9000000	10000	3500
12	5750000	5500	1900
13	6250000	6000	2200
14	5250000	5000	1800
15	6750000	7000	2500
16	4750000	4500	1600
17	7250000	8000	2800
18	4250000	4000	1400
19	5250000	5500	2000
20	8000000	8500	3000
21	5500000	5000	1800
22	8750000	9000	3200
23	7250000	7500	2700
24	6750000	6500	2300

In [109…

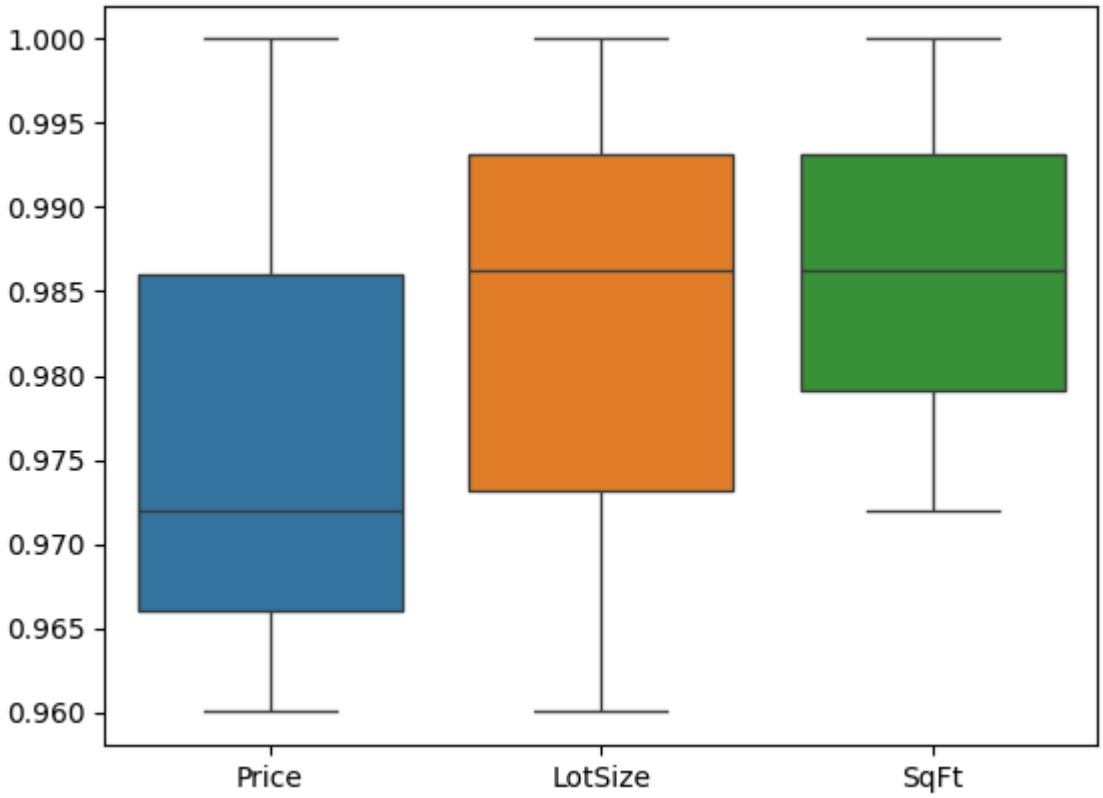
```
s =df.corr()
s
```

Out[109…

	Price	LotSize	SqFt
Price	1.000000	0.960006	0.971984
LotSize	0.960006	1.000000	0.986214
SqFt	0.971984	0.986214	1.000000

In [110…

```
sns.boxplot(s)
plt.show()
```



In [111…

```
# 142

df = pd.read_csv("https://raw.githubusercontent.com/kavit88/Data-Sets/main/house
df
```

Out[111…

	Price	LotSize	SqFt
0	5000000	5000	2000
1	7500000	7500	2500
2	4500000	5500	1800
3	6000000	6500	2200
4	4000000	4500	1600
5	5500000	6000	2000
6	8500000	8000	3000
7	7000000	7000	2500
8	6500000	6500	2400
9	8000000	7500	2800
10	4500000	4500	1500
11	9000000	10000	3500
12	5750000	5500	1900
13	6250000	6000	2200
14	5250000	5000	1800
15	6750000	7000	2500
16	4750000	4500	1600
17	7250000	8000	2800
18	4250000	4000	1400
19	5250000	5500	2000
20	8000000	8500	3000
21	5500000	5000	1800
22	8750000	9000	3200
23	7250000	7500	2700
24	6750000	6500	2300

In [112…

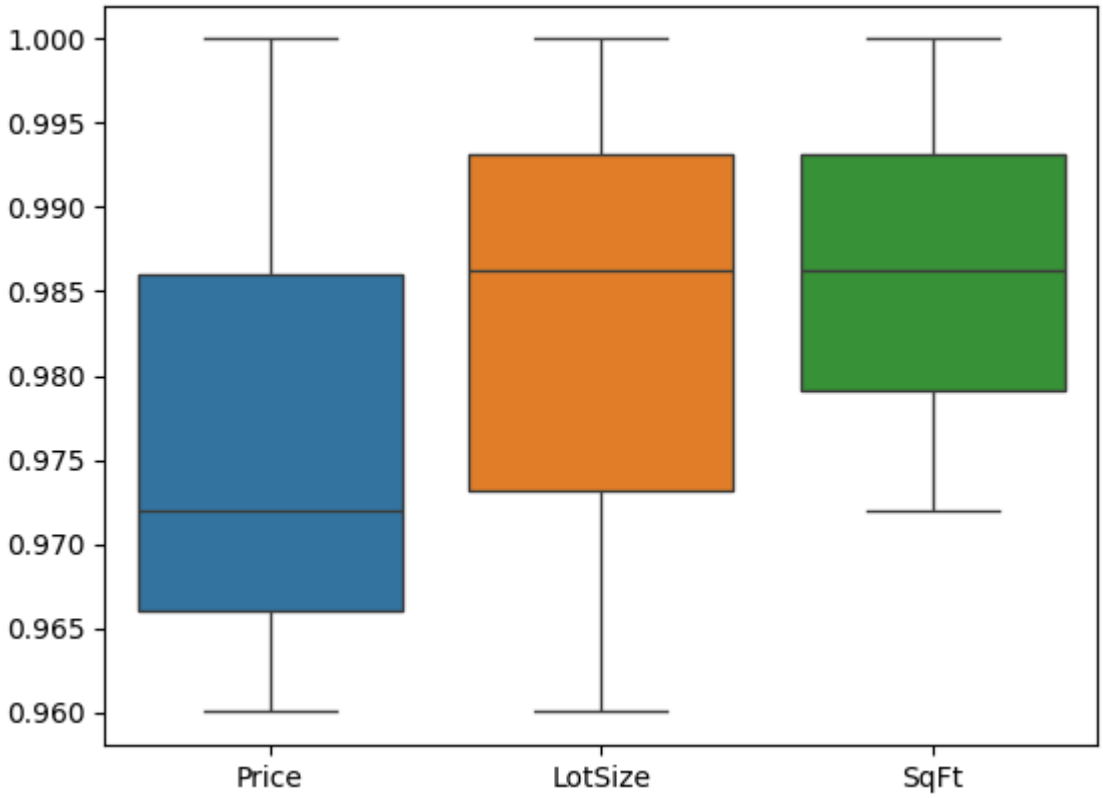
```
df.corr()
```

Out[112…

	Price	LotSize	SqFt
Price	1.000000	0.960006	0.971984
LotSize	0.960006	1.000000	0.986214
SqFt	0.971984	0.986214	1.000000

In [113…

```
sns.boxplot(df.corr())
plt.show()
```



In [114…

```
# 142

df = pd.read_csv("https://raw.githubusercontent.com/kavit88/Data-Sets/main/heigh
df
```

Out[114…

	Height	Weight	Male
0	73.847017	241.893563	1
1	68.781904	162.310473	1
2	74.110105	212.740856	1
3	71.730978	220.042470	1
4	69.881796	206.349801	1
...	...	...	...
9995	66.172652	136.777454	0
9996	67.067155	170.867906	0
9997	63.867992	128.475319	0
9998	69.034243	163.852461	0
9999	61.944246	113.649103	0

10000 rows × 3 columns

In [115…

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Height  10000 non-null  float64
 1   Weight  10000 non-null  float64
 2   Male    10000 non-null  int64
dtypes: float64(2), int64(1)
memory usage: 234.5 KB
```

In [116…

```
df.drop("Male", inplace=True, axis=1)
df.describe()
```

Out[116…

	Height	Weight
count	10000.000000	10000.000000
mean	66.367560	161.440357
std	3.847528	32.108439
min	54.263133	64.700127
25%	63.505620	135.818051
50%	66.318070	161.212928
75%	69.174262	187.169525
max	78.998742	269.989698

In [117…

```
df.corr()
```

Out[117…

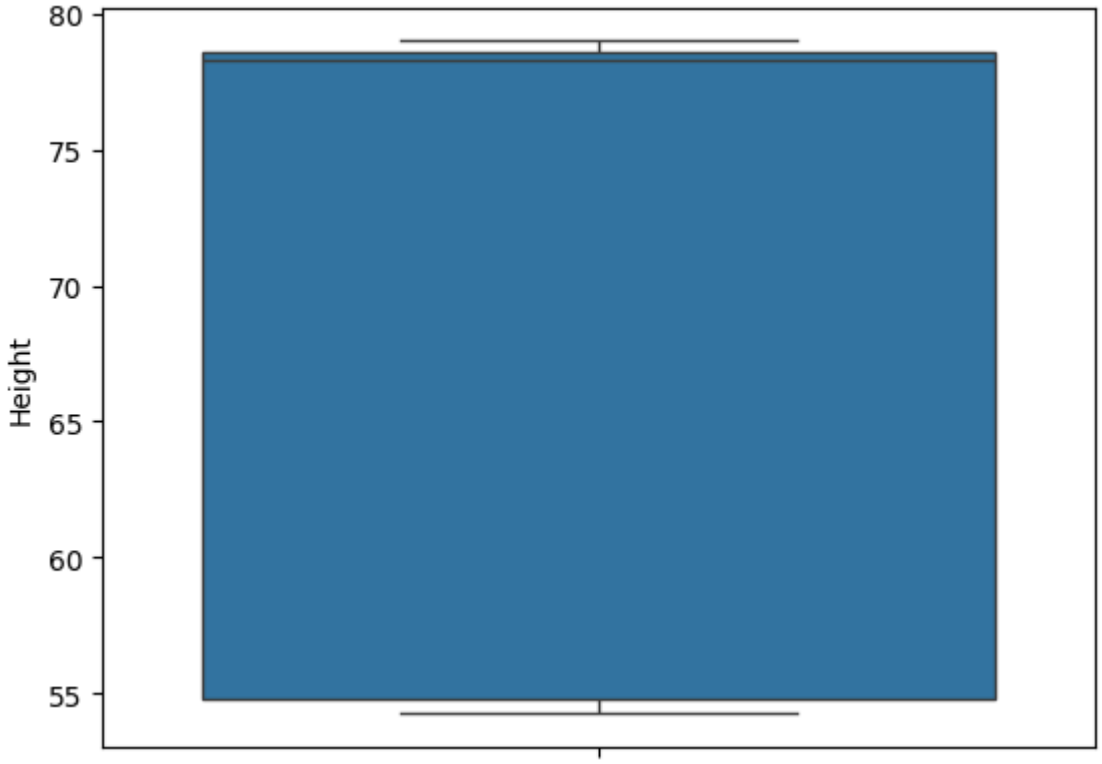
	Height	Weight
Height	1.000000	0.924756
Weight	0.924756	1.000000

In [118…

```
def find_outliers(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = column[(column < lower_bound) | (column > upper_bound)]
    return outliers
```

In [119…

```
sns.boxplot(find_outliers(df["Height"]))
plt.show()
```

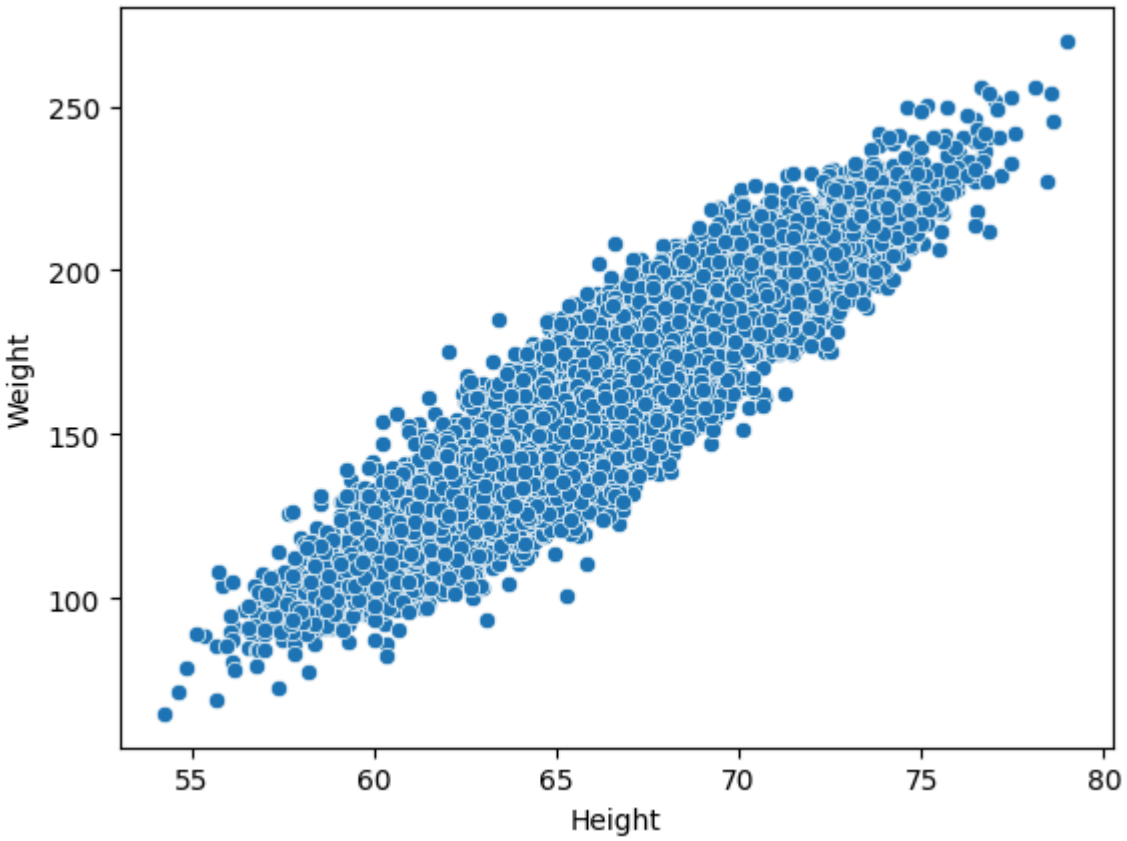


In [120…

```
sns.boxplot(find_outliers(df["Weight"]))
plt.show()
```

In [121…

```
sns.scatterplot(x=df["Height"],y=df["Weight"])
plt.show()
```



In [122…

```
# 143

df =pd.read_csv("https://raw.githubusercontent.com/kavit88/Data-Sets/main/studen
df
```

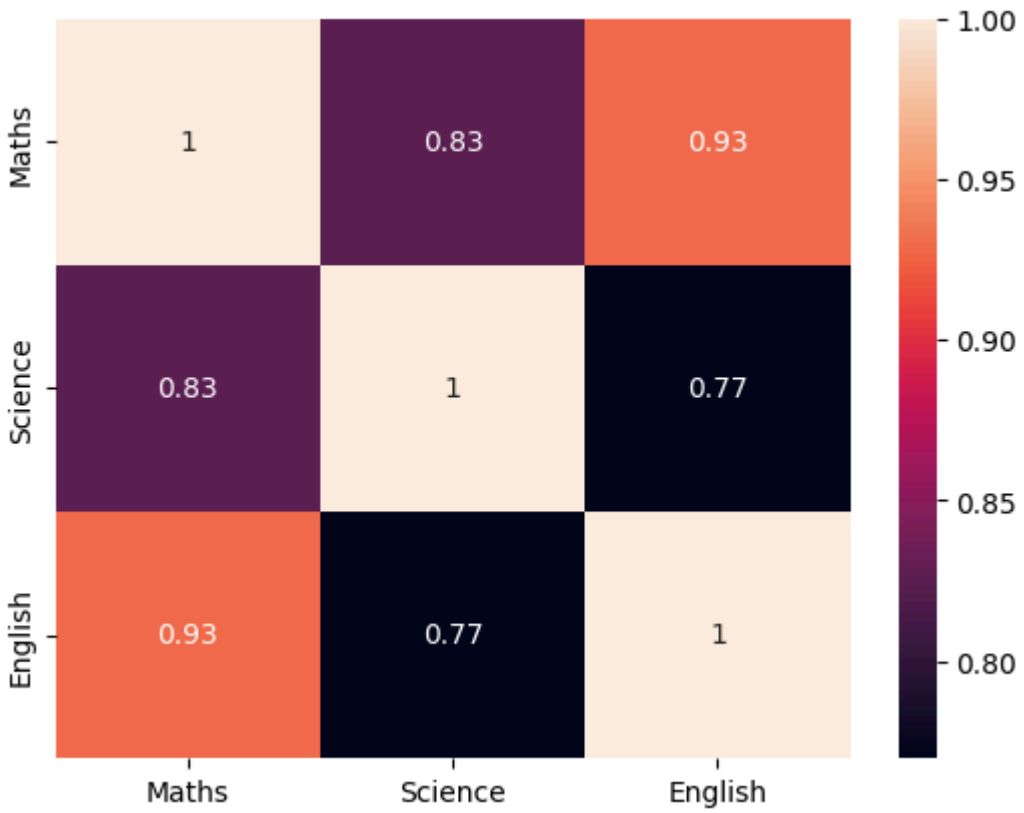
Out[122...

	Name	Maths	Science	English
0	John	85	90	78
1	Emily	92	87	91
2	Jack	76	80	82
3	Alice	89	94	86
4	Tom	78	82	76
5	Lucy	95	96	98
6	Bob	82	85	81
7	Lily	90	92	88
8	Tim	85	78	83
9	Jane	79	84	77
10	Mike	87	88	85
11	Sarah	91	92	93
12	Dave	75	80	76
13	Ann	88	92	89
14	Sam	83	87	81
15	Sue	94	96	97
16	Ben	81	78	84
17	Kim	86	90	88
18	Max	89	92	90
19	Mia	80	84	78
20	Dan	78	80	76
21	Amy	90	92	88
22	Ron	82	85	81
23	Zoe	85	78	83
24	Ted	79	84	77
25	Eva	87	88	85
26	Ray	91	92	93
27	Jay	75	80	76
28	Liv	88	92	89
29	Rob	83	87	81
30	Tim	94	96	97
31	Bea	81	78	84
32	Rex	86	90	88

	Name	Maths	Science	English
33	Meg	89	92	90
34	Jan	80	84	78
35	Ian	78	80	76
36	Mia	90	92	88
37	Ron	82	85	81
38	Zoe	85	78	83
39	Ted	79	84	77
40	Eva	87	88	85
41	Ray	91	92	93
42	Jay	75	80	76
43	Liv	88	92	89
44	Rob	83	87	81

In [123…

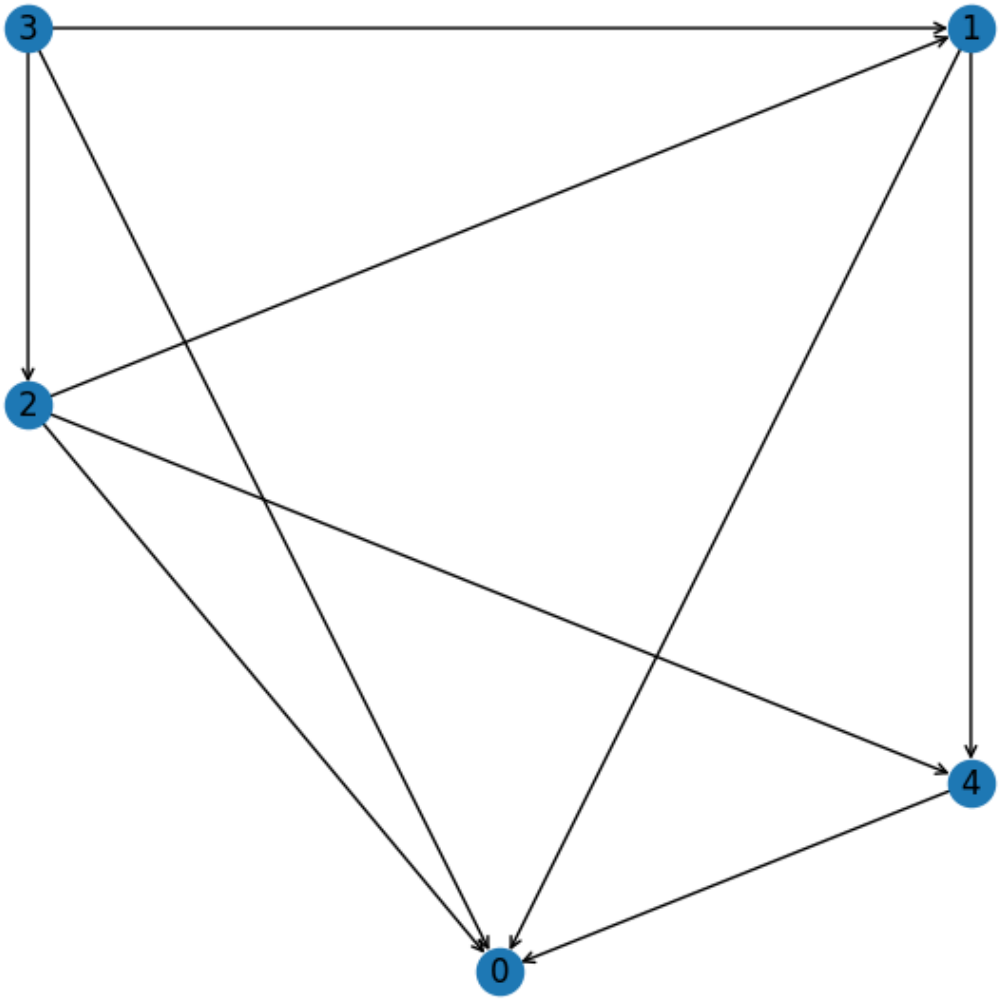
```
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```



In [124…

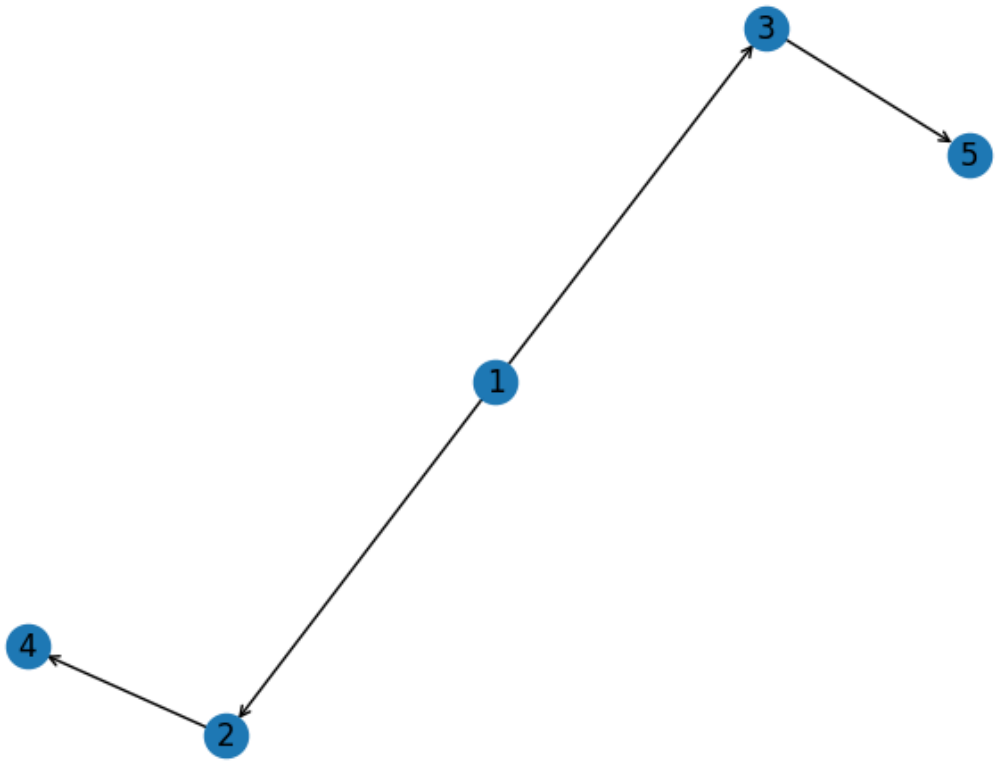
```
# 144

G = nx.DiGraph()
G.add_nodes_from([0, 1, 2, 3, 4])

edges = [
    (3, 1),
```

```
    (3, 2),
    (3, 0),
    (2, 1),
    (2, 4),
    (2, 0),
    (1, 4),
    (1, 0),
    (4, 0)
]

G.add_edges_from(edges)

pos = {
    3: (-1, 1),
    1: (1, 1),
    2: (-1, 0),
    4: (1, -1),
    0: (0, -1.5)
}

# Draw graph
plt.figure(figsize=(6, 6))
nx.draw(
    G, pos,
    with_labels=True,
    arrows=True,
    arrowstyle='->',
)
plt.show()
```



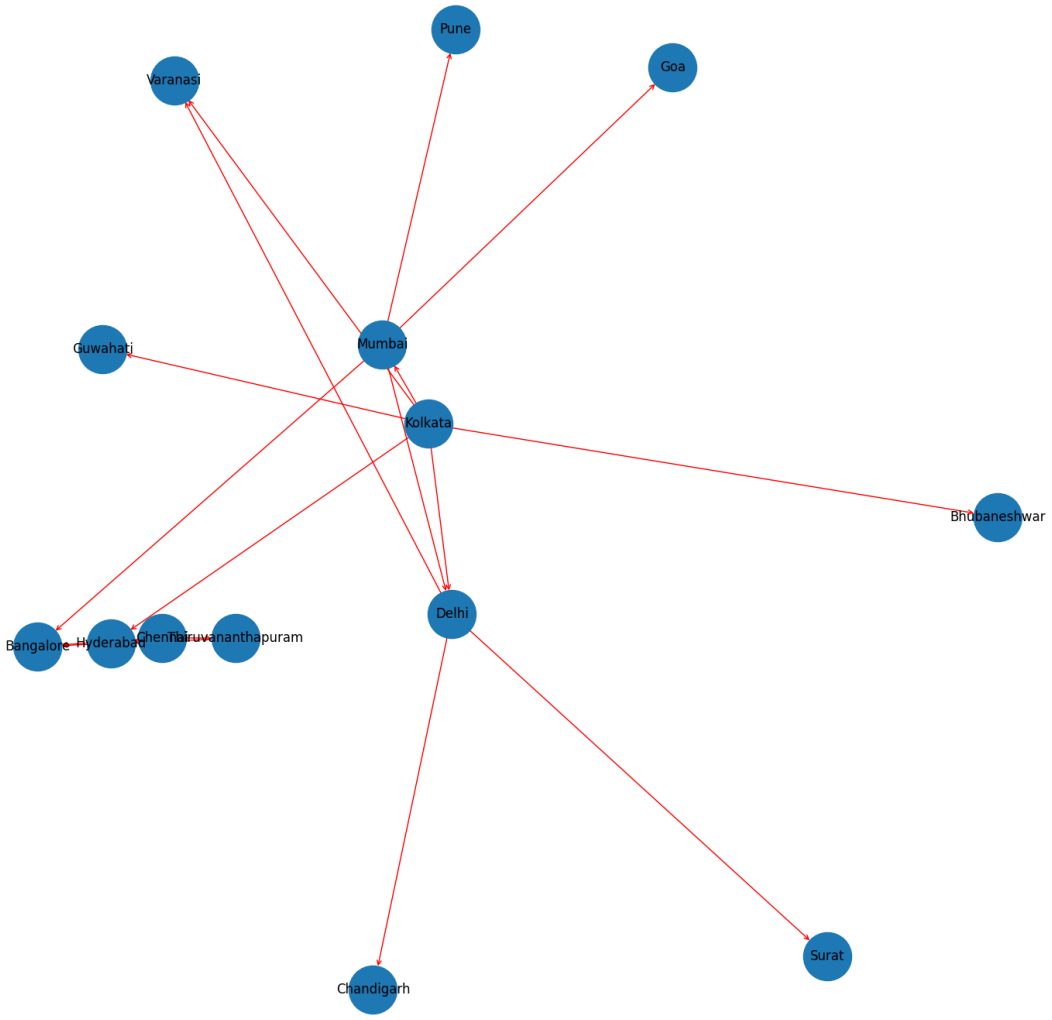
In [125…

```
# 145

G = nx.DiGraph()
G.add_nodes_from([1, 2, 3, 4,5])

edges = [
    (1,2),
    (1,3),
    (2,4),
    (3,5),
    ]
G.add_edges_from(edges)

nx.draw(G, with_labels=True, arrows=True, arrowstyle='->')
plt.show()
```



In [126…

```
# 146

G = nx.DiGraph()

edges = [
    ("Kolkata", "Mumbai"),
    ("Mumbai", "Pune"),
    ("Mumbai", "Goa"),
    ("Kolkata", "Delhi"),
    ("Kolkata", "Bhubaneshwar"),
    ("Mumbai", "Delhi"),
    ("Delhi", "Chandigarh"),
    ("Delhi", "Surat"),
    ("Kolkata", "Hyderabad"),
    ("Hyderabad", "Chennai"),
    ("Chennai", "Thiruvananthapuram"),
    ("Thiruvananthapuram", "Hyderabad"),
    ("Kolkata", "Varanasi"),
    ("Delhi", "Varanasi"),
    ("Mumbai", "Bangalore"),
    ("Chennai", "Bangalore"),
    ("Hyderabad", "Bangalore"),
    ("Kolkata", "Guwahati")
]

G.add_edges_from(edges)

plt.figure(figsize=(15, 15))
nx.draw(G, with_labels=True, arrows=True, arrowstyle='->',node_size=2000, edge_c
```



In [127…

```
# 147

df = pd.read_csv("../datasets/supermarket_sales.csv")
df
```

Out[127…

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.14
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.21
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.28
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.20
...	...	...	...	...	...	...	...	...	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.01
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.59
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.29
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.91

1000 rows × 17 columns

In [128…

```
df.head(8)
```

Out[128…

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085
5	699-14-3026	C	Naypyitaw	Normal	Male	Electronic accessories	85.39	7	29.8865
6	355-53-5943	A	Yangon	Member	Female	Electronic accessories	68.84	6	20.6520
7	315-22-5665	C	Naypyitaw	Normal	Female	Home and lifestyle	73.56	10	36.7800

In [129…

```
df.isna().sum()
```

Out[129…

```
Invoice ID                 0
Branch                     0
City                       0
Customer type              0
Gender                     0
Product line               0
Unit price                 0
Quantity                   0
Tax 5%                     0
Total                      0
Date                       0
Time                       0
Payment                    0
cogs                       4
gross margin percentage    0
gross income               0
Rating                     5
dtype: int64
```

In [130…

```
s =df["Rating"].mean()
df.fillna(s, inplace=True)
```

Out[130…

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.14
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.21
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.28
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.20
...	...	...	...	...	...	...	...	...	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.01
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.59
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.29
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.91

1000 rows × 17 columns

In [131…

```
df[(df["Quantity"]>3) & ((df['Rating']>8.5) | (df["Total"]>600))].shape[0]
```

Out[131…

```
295
```

In [132…

```
df.groupby("Customer type")["Total"].agg("sum")
```

Out[132…

```
Customer type
Member    164223.444
Normal    158743.305
Name: Total, dtype: float64
```

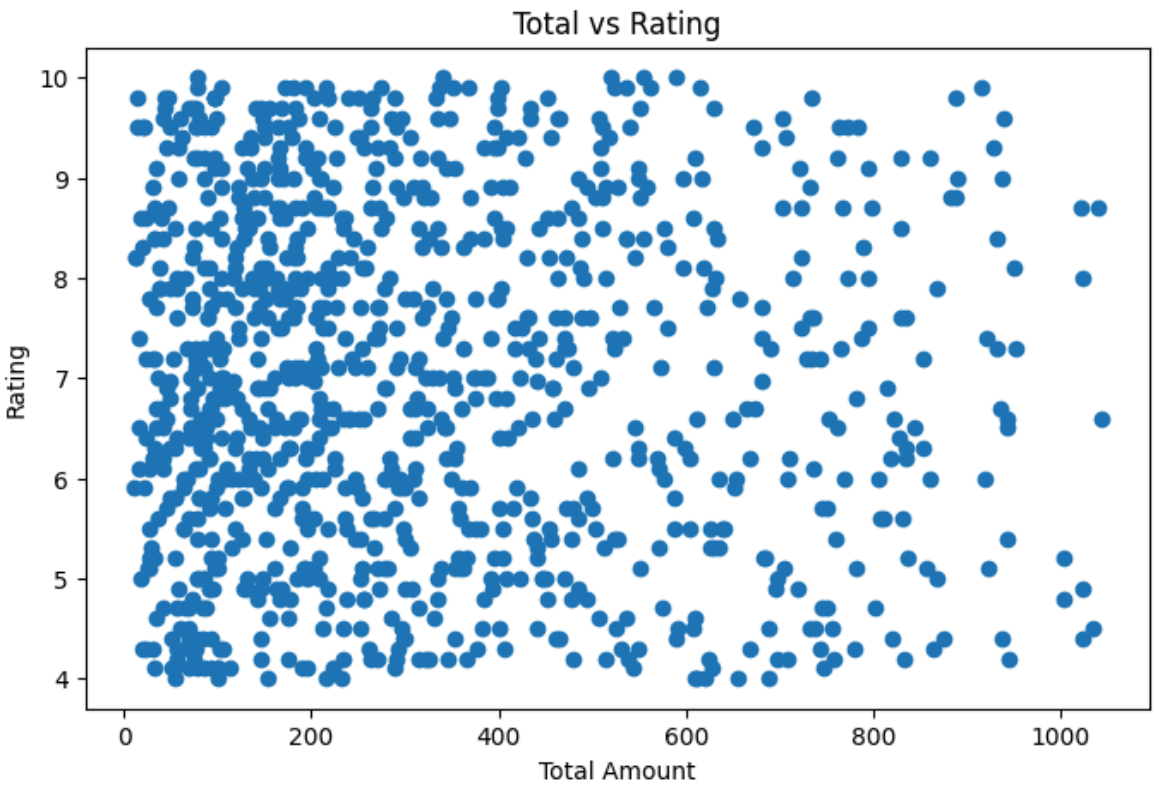
In [133…
```
new=df.groupby('Payment')['gross income'].sum()
Q5=(new/new.sum())*100
Q5
```

Out[133…
```
Payment
Cash           34.742453
Credit card    31.200448
Ewallet        34.057099
Name: gross income, dtype: float64
```

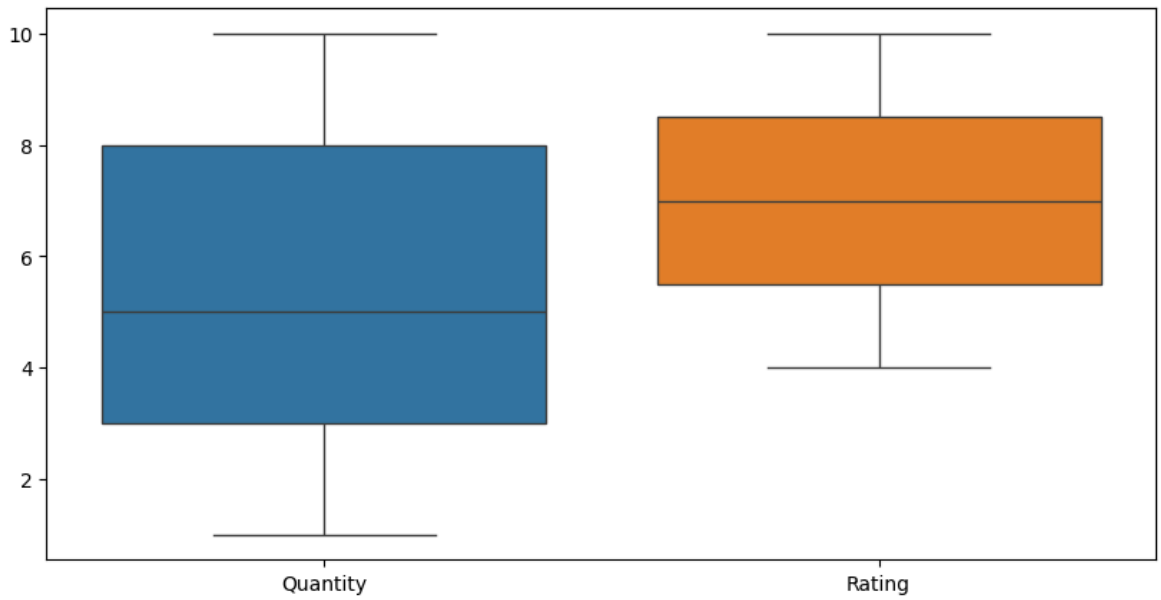
In [134…
```
df.groupby("Gender")["Total"].mean()
```

Out[134…
```
Gender
Female    335.095659
Male      310.789226
Name: Total, dtype: float64
```

In [135…
```
plt.figure(figsize=(8,5))
plt.scatter(df['Total'], df['Rating'])
plt.xlabel('Total Amount')
plt.ylabel('Rating')
plt.title('Total vs Rating')
plt.show()
```

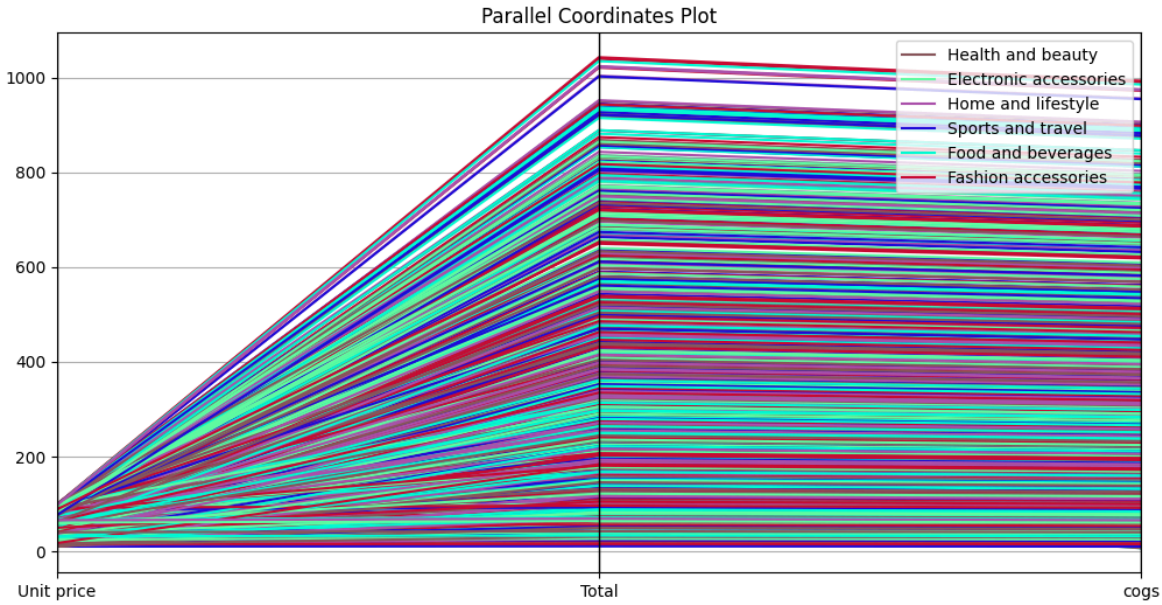


In [136…
```
plt.figure(figsize=(10,5))

sns.boxplot([df['Quantity'],df['Rating']])
plt.show()
```



In [137…

```
df_pc = df[['Product line', 'Unit price', 'Total', 'cogs']]

plt.figure(figsize=(12,6))
pd.plotting.parallel_coordinates(df_pc, 'Product line')
plt.title('Parallel Coordinates Plot')
plt.show()
```



In [138…

```
# 148
df = pd.read_csv("../datasets/data.csv")
df
```

Out[138…

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
...	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.0
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

169 rows × 4 columns

In [139…

```
df.describe()
```

Out[139…

	Duration	Pulse	Maxpulse	Calories
count	169.000000	169.000000	169.000000	164.000000
mean	63.846154	107.461538	134.047337	375.790244
std	42.299949	14.510259	16.450434	266.379919
min	15.000000	80.000000	100.000000	50.300000
25%	45.000000	100.000000	124.000000	250.925000
50%	60.000000	105.000000	131.000000	318.600000
75%	60.000000	111.000000	141.000000	387.600000
max	300.000000	159.000000	184.000000	1860.400000

In [140…

```
df.corr(numeric_only=True)
```

Out[140…

	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922717
Pulse	-0.155408	1.000000	0.786535	0.025121
Maxpulse	0.009403	0.786535	1.000000	0.203813
Calories	0.922717	0.025121	0.203813	1.000000

In [141…

```
df.isna().sum()
```

Out[141…

```
Duration    0
Pulse       0
Maxpulse    0
Calories    5
dtype: int64
```

In [142…

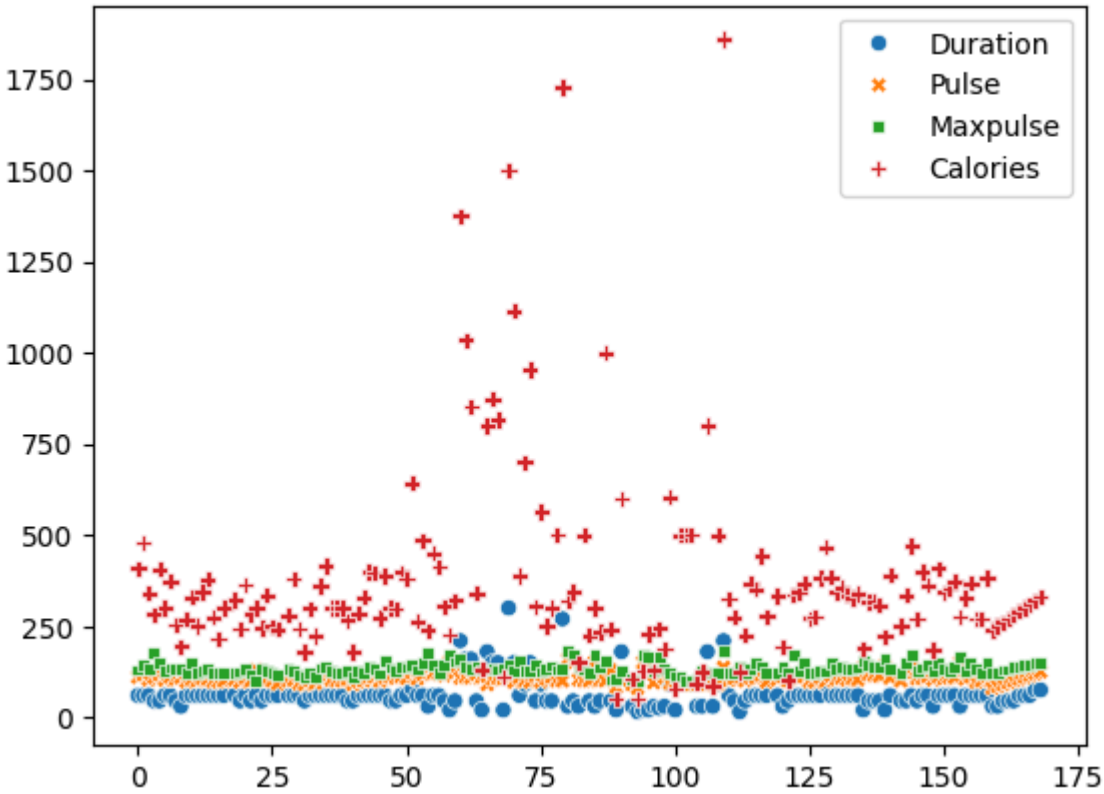
```
df.dropna(inplace=True)
df.isna().sum()
df.shape
```

Out[142…

```
(164, 4)
```

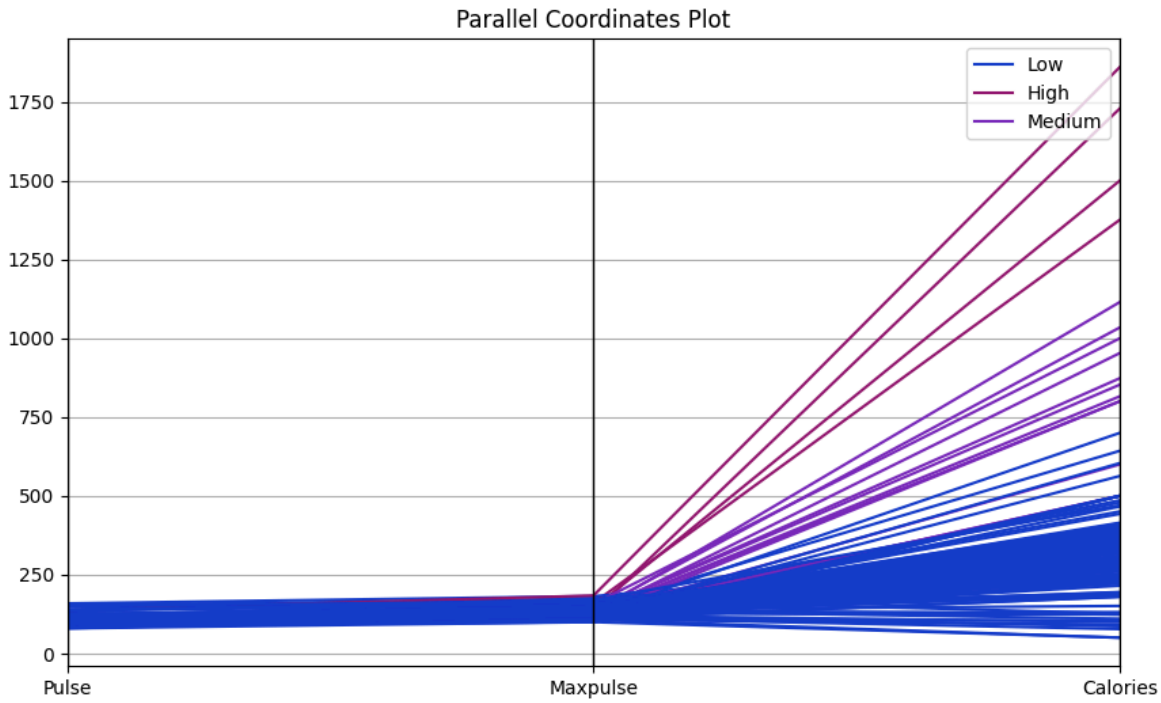
In [143…

```
sns.scatterplot(df)
plt.show()
```



In [144…

```
df['Duration_category'] = pd.cut(df['Duration'], bins=3, labels=['Low', 'Medium'

plt.figure(figsize=(10,6))
pd.plotting.parallel_coordinates(
    df[['Duration_category','Pulse','Maxpulse','Calories']],
    'Duration_category'
)
plt.title("Parallel Coordinates Plot")
plt.show()
```



In [145…

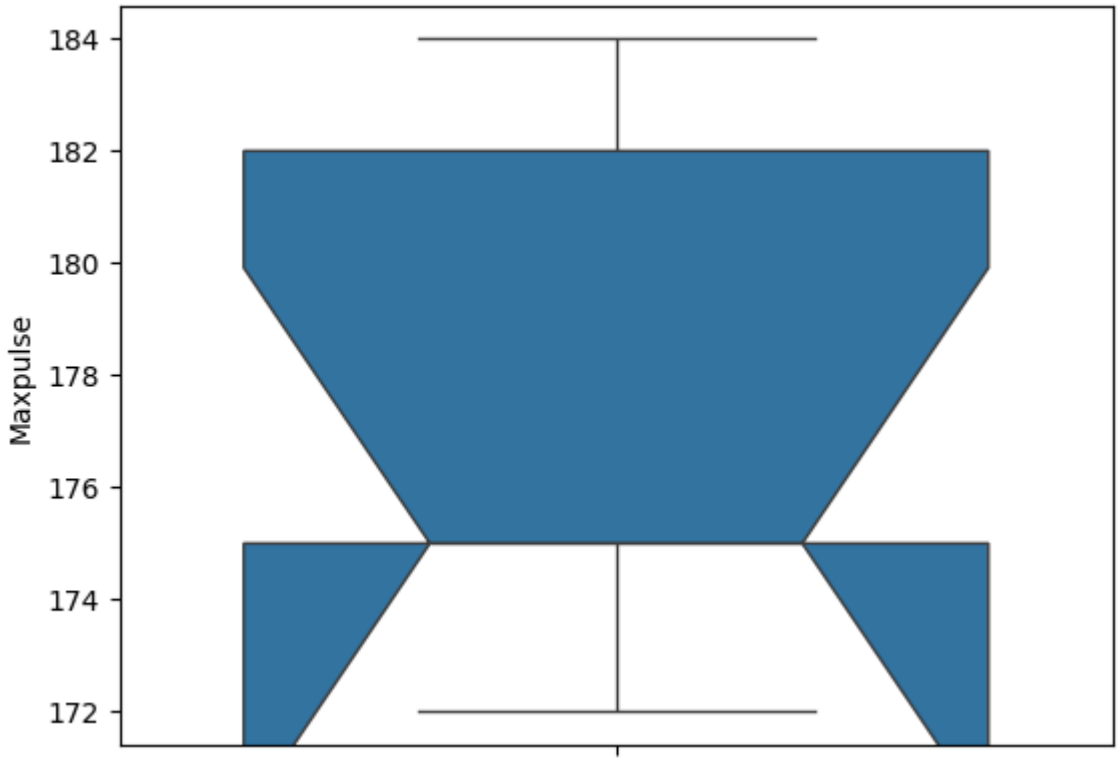
```
def find_outliers(x):
    Q1=x.quantile(0.25)
    Q3=x.quantile(0.75)
    IQR=Q3-Q1
    lower_bound=Q1-1.5*IQR
    upper_bound=Q3+1.5*IQR
    return x[(x<lower_bound) | (x>upper_bound)]

find_outliers(df["Maxpulse"])
```

Out[145…

```
3      175
54     175
58     172
80     182
109    184
Name: Maxpulse, dtype: int64
```

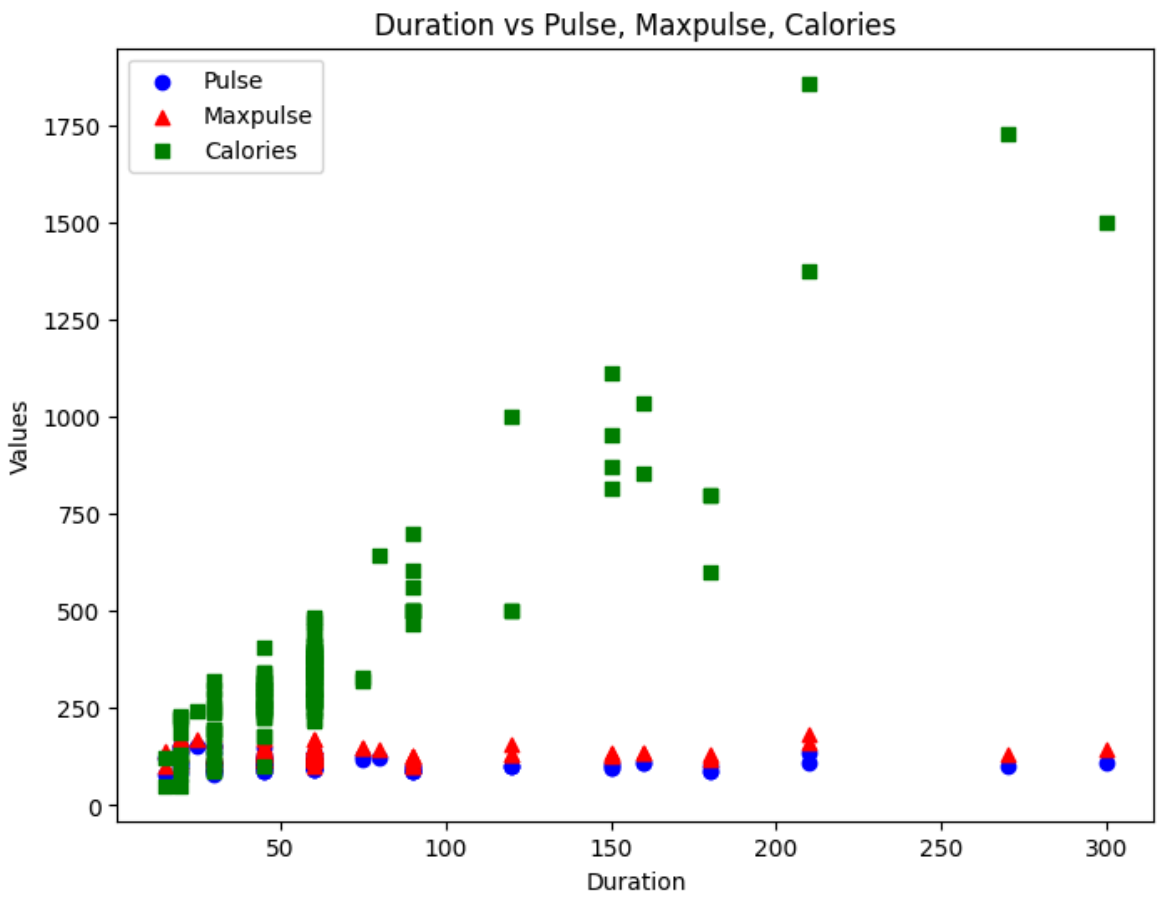
In [146…

```
sns.boxplot(find_outliers(df["Maxpulse"]),widths=0.75,notch=True)
plt.show()
```



In [147…

```
plt.figure(figsize=(8,6))

plt.scatter(df['Duration'], df['Pulse'],
            color='blue', marker='o', label='Pulse')

plt.scatter(df['Duration'], df['Maxpulse'],
            color='red', marker='^', label='Maxpulse')

plt.scatter(df['Duration'], df['Calories'],
            color='green', marker='s', label='Calories')

plt.xlabel('Duration')
plt.ylabel('Values')
plt.title('Duration vs Pulse, Maxpulse, Calories')
plt.legend()
plt.show()
```



In [148…

```
df = pd.read_csv("../datasets/Iris.csv")
df
```

Out[148…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [149…

```
df.head()
```

Out[149…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [150…

```
df.dtypes
```

Out[150…

```
Id                 int64
SepalLengthCm    float64
SepalWidthCm     float64
PetalLengthCm    float64
PetalWidthCm     float64
Species              str
dtype: object
```

In [151…

```
df.isnull().sum()
```

Out[151…

```
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

In [152…

```
df.tail()
```

Out[152…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

In [153…

```
df.corr(numeric_only=True)
```

Out[153…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthC
Id	1.000000	0.716676	-0.397729	0.882747	0.8997
SepalLengthCm	0.716676	1.000000	-0.109369	0.871754	0.8179
SepalWidthCm	-0.397729	-0.109369	1.000000	-0.420516	-0.3565
PetalLengthCm	0.882747	0.871754	-0.420516	1.000000	0.9627
PetalWidthCm	0.899759	0.817954	-0.356544	0.962757	1.0000

In [154…

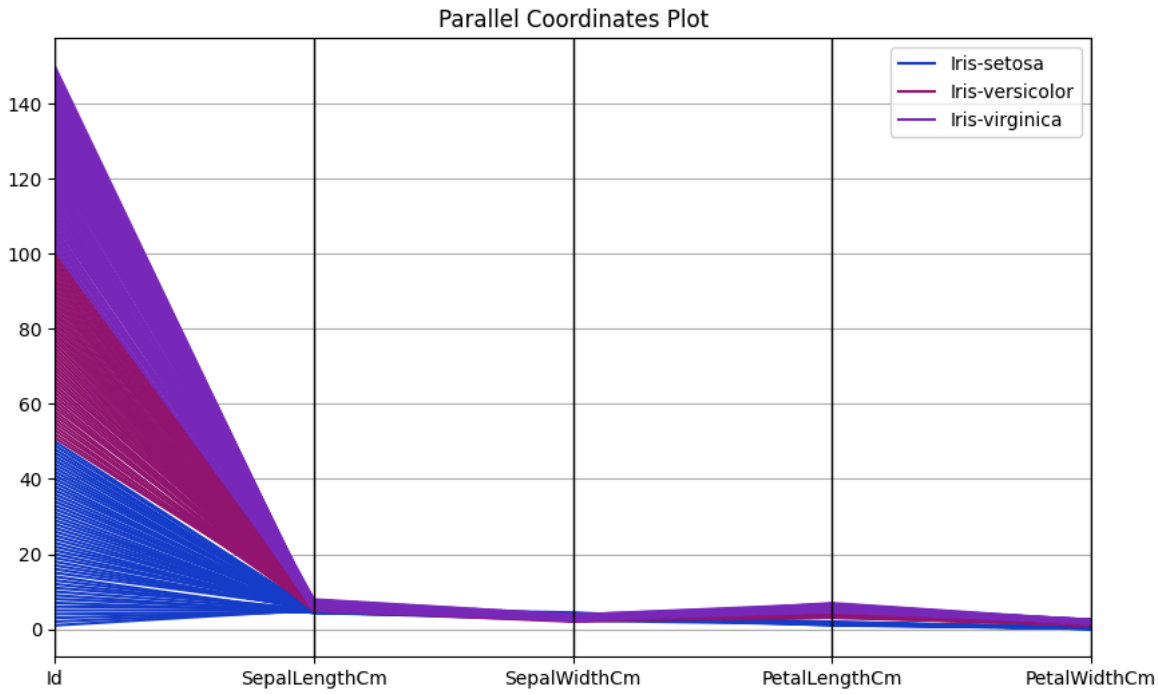
```
df.median(numeric_only=True)
```

Out[154…

```
Id               75.50
SepalLengthCm     5.80
SepalWidthCm      3.00
PetalLengthCm     4.35
PetalWidthCm      1.30
dtype: float64
```

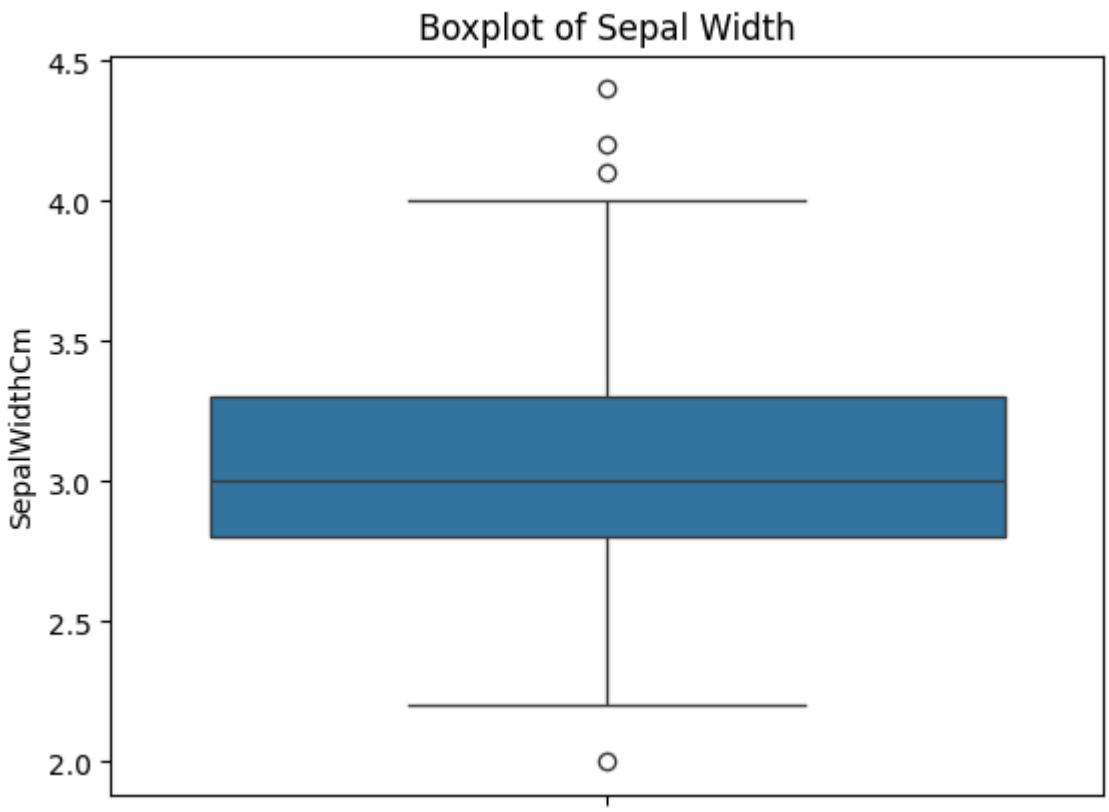
In [155…

```
plt.figure(figsize=(10,6))
pd.plotting.parallel_coordinates(df, 'Species')
plt.title('Parallel Coordinates Plot')
plt.show()
```



In [156…

```
sns.boxplot(df['SepalWidthCm'])
plt.title("Boxplot of Sepal Width")
plt.show()
```



In [157…

```
pd.crosstab(df["SepalLengthCm"], df["PetalWidthCm"])
```

Out[157…

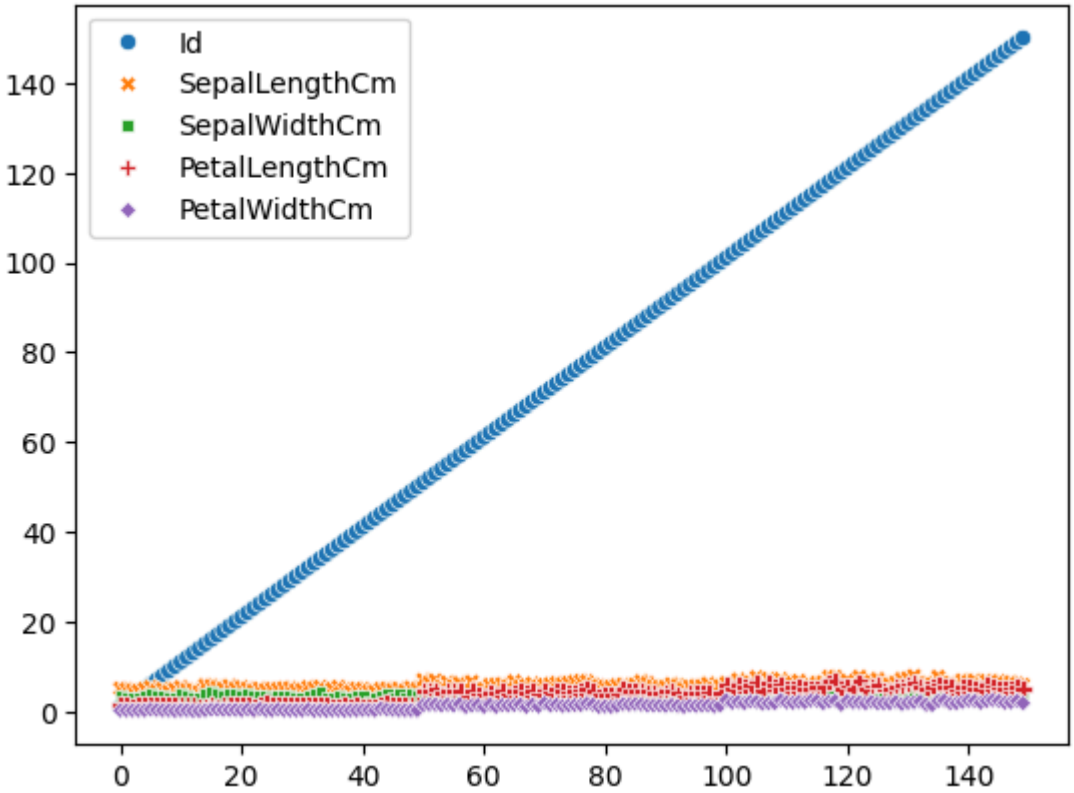
PetalWidthCm	0.1	0.2	0.3	0.4	0.5	0.6	1.0	1.1	1.2	1.3	…	1.6	1.7	1.8	1.9
SepalLengthCm															
4.3	1	0	0	0	0	0	0	0	0	0	…	0	0	0	0
4.4	0	3	0	0	0	0	0	0	0	0	…	0	0	0	0
4.5	0	0	1	0	0	0	0	0	0	0	…	0	0	0	0
4.6	0	3	1	0	0	0	0	0	0	0	…	0	0	0	0
4.7	0	2	0	0	0	0	0	0	0	0	…	0	0	0	0
4.8	1	3	1	0	0	0	0	0	0	0	…	0	0	0	0
4.9	3	1	0	0	0	0	1	0	0	0	…	0	1	0	0
5.0	0	5	1	1	0	1	2	0	0	0	…	0	0	0	0
5.1	0	3	2	2	1	0	0	1	0	0	…	0	0	0	0
5.2	1	2	0	0	0	0	0	0	0	0	…	0	0	0	0
5.3	0	1	0	0	0	0	0	0	0	0	…	0	0	0	0
5.4	0	2	0	3	0	0	0	0	0	0	…	0	0	0	0
5.5	0	2	0	0	0	0	1	1	1	2	…	0	0	0	0
5.6	0	0	0	0	0	0	0	1	0	3	…	0	0	0	0
5.7	0	0	1	1	0	0	1	0	1	3	…	0	0	0	0
5.8	0	1	0	0	0	0	1	0	2	0	…	0	0	0	2
5.9	0	0	0	0	0	0	0	0	0	0	…	0	0	2	0
6.0	0	0	0	0	0	0	1	0	0	0	…	2	0	1	0
6.1	0	0	0	0	0	0	0	0	1	1	…	0	0	1	0
6.2	0	0	0	0	0	0	0	0	0	1	…	0	0	1	0
6.3	0	0	0	0	0	0	0	0	0	1	…	1	0	2	1
6.4	0	0	0	0	0	0	0	0	0	1	…	0	0	1	1
6.5	0	0	0	0	0	0	0	0	0	0	…	0	0	1	0
6.6	0	0	0	0	0	0	0	0	0	1	…	0	0	0	0
6.7	0	0	0	0	0	0	0	0	0	0	…	0	1	1	0
6.8	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0
6.9	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0
7.0	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0
7.1	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0
7.2	0	0	0	0	0	0	0	0	0	0	…	1	0	1	0
7.3	0	0	0	0	0	0	0	0	0	0	…	0	0	1	0
7.4	0	0	0	0	0	0	0	0	0	0	…	0	0	0	1

PetalWidthCm	0.1	0.2	0.3	0.4	0.5	0.6	1.0	1.1	1.2	1.3	...	1.6	1.7	1.8	1.9
SepalLengthCm															
7.6	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
7.7	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
7.9	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0

35 rows × 22 columns

In [158…

```
sns.scatterplot(df)
plt.show()
```



In [159…

```
df["SepalLengthSize"]= df["SepalLengthCm"].apply(lambda x: 'Short' if x < 5 else
df
```

Out[159…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Sep
0	1	5.1	3.5	1.4	0.2	Iris-setosa	
1	2	4.9	3.0	1.4	0.2	Iris-setosa	
2	3	4.7	3.2	1.3	0.2	Iris-setosa	
3	4	4.6	3.1	1.5	0.2	Iris-setosa	
4	5	5.0	3.6	1.4	0.2	Iris-setosa	
...	...	...	...	...	...	...	
145	146	6.7	3.0	5.2	2.3	Iris-virginica	
146	147	6.3	2.5	5.0	1.9	Iris-virginica	
147	148	6.5	3.0	5.2	2.0	Iris-virginica	
148	149	6.2	3.4	5.4	2.3	Iris-virginica	
149	150	5.9	3.0	5.1	1.8	Iris-virginica	

150 rows × 7 columns

In [160…

```
df =pd.read_csv("../datasets/diabetes_unclean.csv")
df
```

Out[160…

	ID	No_Pation	Gender	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLDL
0	502	17975	F	50.0	4.7	46.0	4.9	4.2	0.9	2.4	1.4	0.5
1	735	34221	M	26.0	4.5	62.0	4.9	3.7	1.4	1.1	2.1	0.6
2	420	47975	F	50.0	4.7	46.0	4.9	4.2	0.9	2.4	1.4	0.5
3	680	87656	F	50.0	4.7	46.0	4.9	4.2	0.9	2.4	1.4	0.5
4	504	34223	M	33.0	7.1	46.0	4.9	4.9	1.0	0.8	2.0	0.4
...	...	...	...	...	...	...	...	...	...	...	...	...
1004	191	454316	M	55.0	NaN	62.0	6.8	5.3	2.0	1.0	3.5	0.9
1005	192	454316	M	55.0	4.8	88.0	NaN	5.7	4.0	0.9	3.3	1.8
1006	193	454316	M	62.0	6.3	82.0	6.7	5.3	2.0	1.0	3.5	NaN
1007	194	454316	F	57.0	4.1	70.0	9.3	5.3	3.3	1.0	1.4	1.3
1008	195	4543	f	55.0	4.1	34.0	13.9	5.4	1.6	1.6	3.1	0.7

1009 rows × 14 columns

In [161…

```
df.describe(include=['str'])
```

Out[161…

	Gender	CLASS
count	1009	1009
unique	3	5
top	M	Y
freq	570	840

In [162…

```
df.describe()
```

Out[162…

	ID	No_Pation	AGE	Urea	Cr	HbA1c
count	1009.000000	1.009000e+03	1008.000000	1008.000000	1007.000000	1006.000000
mean	339.161546	2.717448e+05	53.620040	5.131094	68.973188	8.284155
std	239.738169	3.365681e+06	8.740975	2.931136	59.813297	2.533576
min	1.000000	1.230000e+02	25.000000	0.500000	6.000000	0.900000
25%	127.000000	2.406500e+04	51.000000	3.700000	48.000000	6.500000
50%	296.000000	3.439900e+04	55.000000	4.600000	60.000000	8.000000
75%	548.000000	4.539000e+04	59.000000	5.700000	73.000000	10.200000
max	800.000000	7.543566e+07	79.000000	38.900000	800.000000	16.000000

In [163…

```
df.shape
```

Out[163… (1009, 14)

In [164…
```
df.isnull().sum()
```

Out[164…
```
ID           0
No_Pation    0
Gender       0
AGE          1
Urea         1
Cr           2
HbA1c        3
Chol         2
TG           2
HDL          1
LDL          2
VLDL         1
BMI          0
CLASS        0
dtype: int64
```

In [165…
```
df["HbA1c"] = df["HbA1c"].fillna(df["HbA1c"].mean())
df
```

Out[165…

	ID	No_Pation	Gender	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLD
0	502	17975	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0.
1	735	34221	M	26.0	4.5	62.0	4.900000	3.7	1.4	1.1	2.1	0.
2	420	47975	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0.
3	680	87656	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0.
4	504	34223	M	33.0	7.1	46.0	4.900000	4.9	1.0	0.8	2.0	0.
...	...	...	...	...	...	...	...	...	...	...	...	.
1004	191	454316	M	55.0	NaN	62.0	6.800000	5.3	2.0	1.0	3.5	0.
1005	192	454316	M	55.0	4.8	88.0	8.284155	5.7	4.0	0.9	3.3	1.
1006	193	454316	M	62.0	6.3	82.0	6.700000	5.3	2.0	1.0	3.5	NaN
1007	194	454316	F	57.0	4.1	70.0	9.300000	5.3	3.3	1.0	1.4	1.
1008	195	4543	f	55.0	4.1	34.0	13.900000	5.4	1.6	1.6	3.1	0.

1009 rows × 14 columns

In [166…
```
df.dropna(inplace=True)
df
```

Out[166…

	ID	No_Pation	Gender	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLI
0	502	17975	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0
1	735	34221	M	26.0	4.5	62.0	4.900000	3.7	1.4	1.1	2.1	0
2	420	47975	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0
3	680	87656	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0
4	504	34223	M	33.0	7.1	46.0	4.900000	4.9	1.0	0.8	2.0	0
...	...	...	...	...	...	...	...	...	...	...	...	
1002	188	454316	F	75.0	10.3	113.0	8.600000	4.2	1.6	0.9	2.6	0
1003	189	454316	M	58.0	4.0	55.0	7.900000	4.9	2.0	1.2	1.4	1
1005	192	454316	M	55.0	4.8	88.0	8.284155	5.7	4.0	0.9	3.3	1
1007	194	454316	F	57.0	4.1	70.0	9.300000	5.3	3.3	1.0	1.4	1
1008	195	4543	f	55.0	4.1	34.0	13.900000	5.4	1.6	1.6	3.1	0

997 rows × 14 columns

In [167…

```
df.corr(numeric_only=True)
```

Out[167…

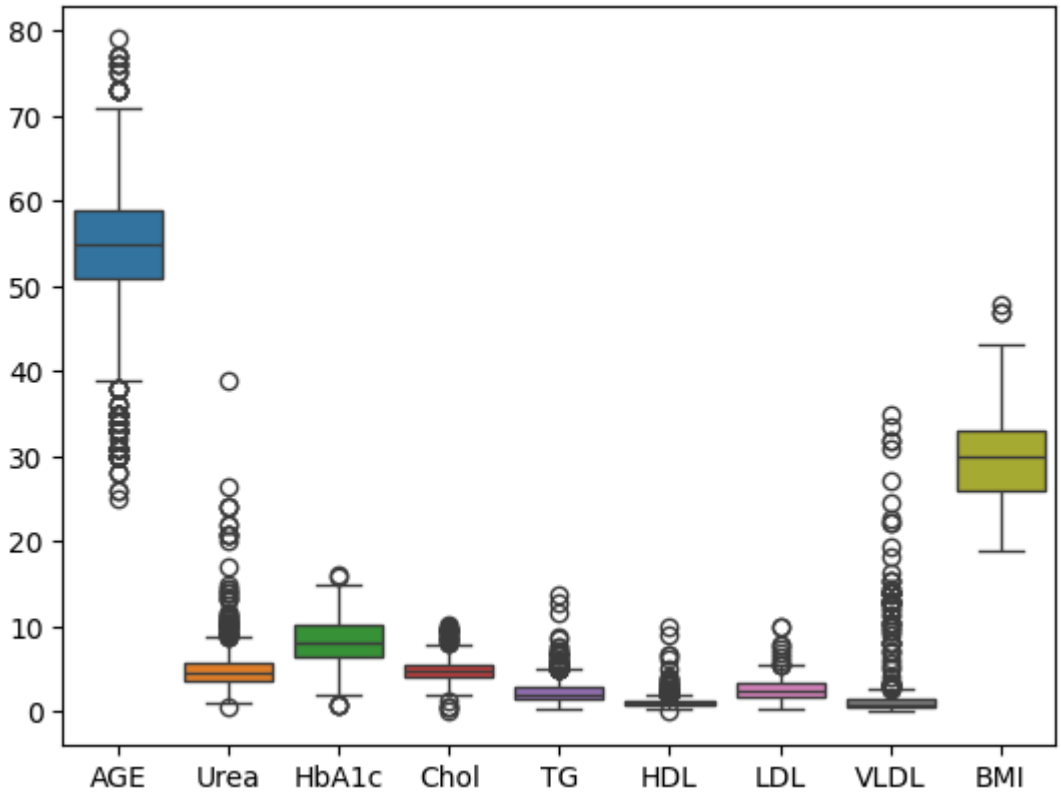
	ID	No_Pation	AGE	Urea	Cr	HbA1c	Chol
ID	1.000000	0.064599	-0.072687	-0.094891	-0.100046	-0.009037	0.045414
No_Pation	0.064599	1.000000	-0.088870	-0.019061	0.000973	-0.032350	-0.030288
AGE	-0.072687	-0.088870	1.000000	0.108613	0.056940	0.384675	0.038966
Urea	-0.094891	-0.019061	0.108613	1.000000	0.624810	-0.023307	0.001286
Cr	-0.100046	0.000973	0.056940	0.624810	1.000000	-0.037735	-0.007636
HbA1c	-0.009037	-0.032350	0.384675	-0.023307	-0.037735	1.000000	0.177676
Chol	0.045414	-0.030288	0.038966	0.001286	-0.007636	0.177676	1.000000
TG	-0.054110	-0.039859	0.149274	0.040939	0.056031	0.214030	0.318894
HDL	0.025226	-0.013554	-0.021029	-0.037843	-0.023578	0.030455	0.103370
LDL	-0.065718	-0.003520	0.011496	-0.006673	0.040981	0.011536	0.419237
VLDL	0.145700	0.113635	-0.090796	-0.011573	0.010328	0.072641	0.076373
BMI	0.047270	0.017738	0.381176	0.045753	0.054847	0.413130	0.016989

In [168…

```
sns.boxplot( [df['AGE'], df['Urea'],df['HbA1c'],df['Chol'],df['TG'],df['HDL'],df
```

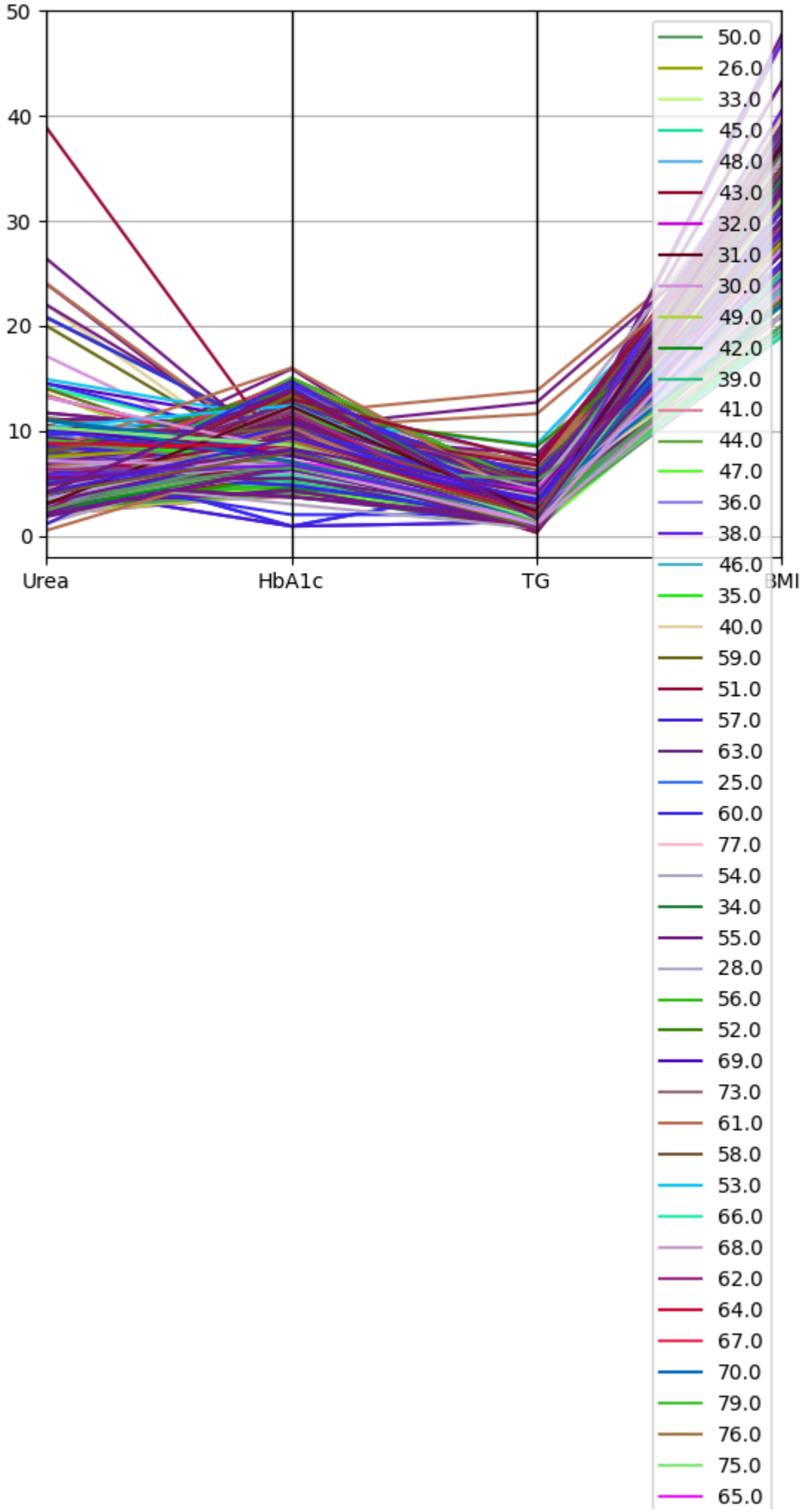
Out[168…

```
<Axes: >
```



In [169…

```
pd.plotting.parallel_coordinates(df[["Urea", "HbA1c", "TG", "BMI", "AGE"]], "AGE
plt.show()
```


50
40
30
20
10
0
Urea
HbA1c
TG
BMI
50.0
26.0
33.0
45.0
48.0
43.0
32.0
31.0
30.0
49.0
42.0
39.0
41.0
44.0
47.0
36.0
38.0
46.0
35.0
40.0
59.0
51.0
57.0
63.0
25.0
60.0
77.0
54.0
34.0
55.0
28.0
56.0
52.0
69.0
73.0
61.0
58.0
53.0
66.0
68.0
62.0
64.0
67.0
70.0
79.0
76.0
75.0
65.0

71.0

In [170…
```
df.drop(df[df["Gender"] == "f"].index, inplace=True)
df
```

Out[170…

	ID	No_Pation	Gender	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLD
0	502	17975	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0.
1	735	34221	M	26.0	4.5	62.0	4.900000	3.7	1.4	1.1	2.1	0.
2	420	47975	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0.
3	680	87656	F	50.0	4.7	46.0	4.900000	4.2	0.9	2.4	1.4	0.
4	504	34223	M	33.0	7.1	46.0	4.900000	4.9	1.0	0.8	2.0	0.
...	...	...	...	...	...	...	...	...	...	...	...	.
1000	185	454316	M	64.0	8.8	106.0	8.500000	5.9	2.1	1.2	4.0	1.
1002	188	454316	F	75.0	10.3	113.0	8.600000	4.2	1.6	0.9	2.6	0.
1003	189	454316	M	58.0	4.0	55.0	7.900000	4.9	2.0	1.2	1.4	1.
1005	192	454316	M	55.0	4.8	88.0	8.284155	5.7	4.0	0.9	3.3	1.
1007	194	454316	F	57.0	4.1	70.0	9.300000	5.3	3.3	1.0	1.4	1.

995 rows × 14 columns

In [171…
```
df.Gender.value_counts()
```

Out[171…
```
Gender
M    563
F    432
Name: count, dtype: int64
```

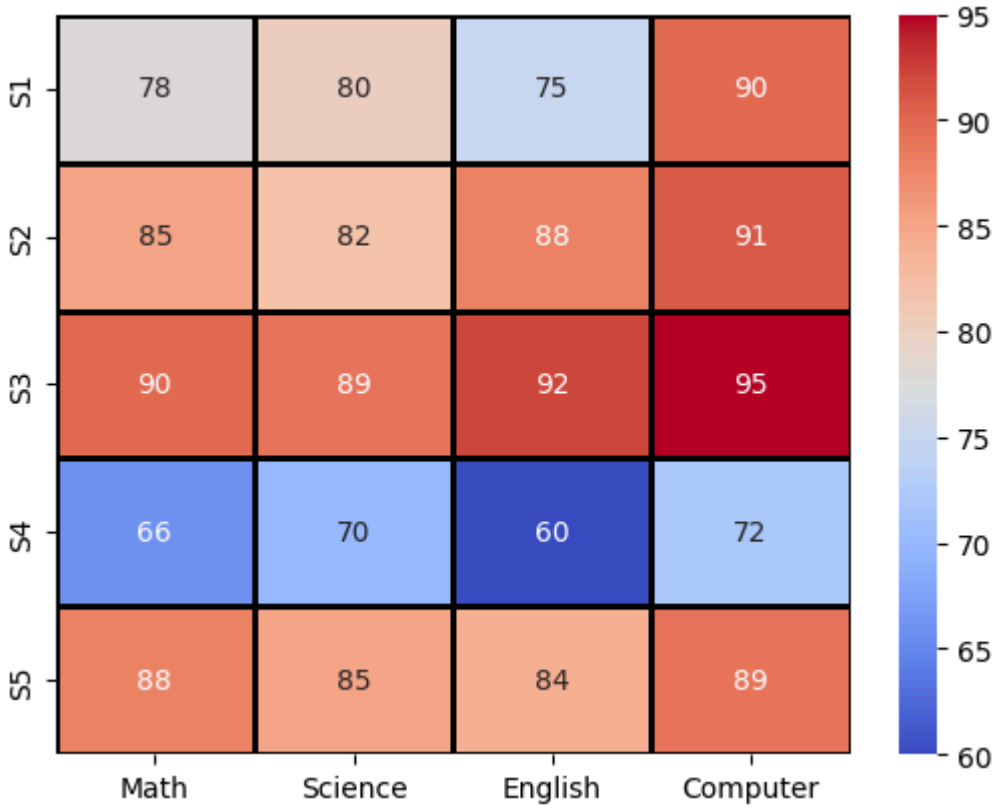
In [172…
```
def remove_outliers(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return column[(column >= lower_bound) & (column <= upper_bound)]
```

In [173…
```
remove_outliers(df["HbA1c"]).shape
```

Out[173…
```
(989,)
```

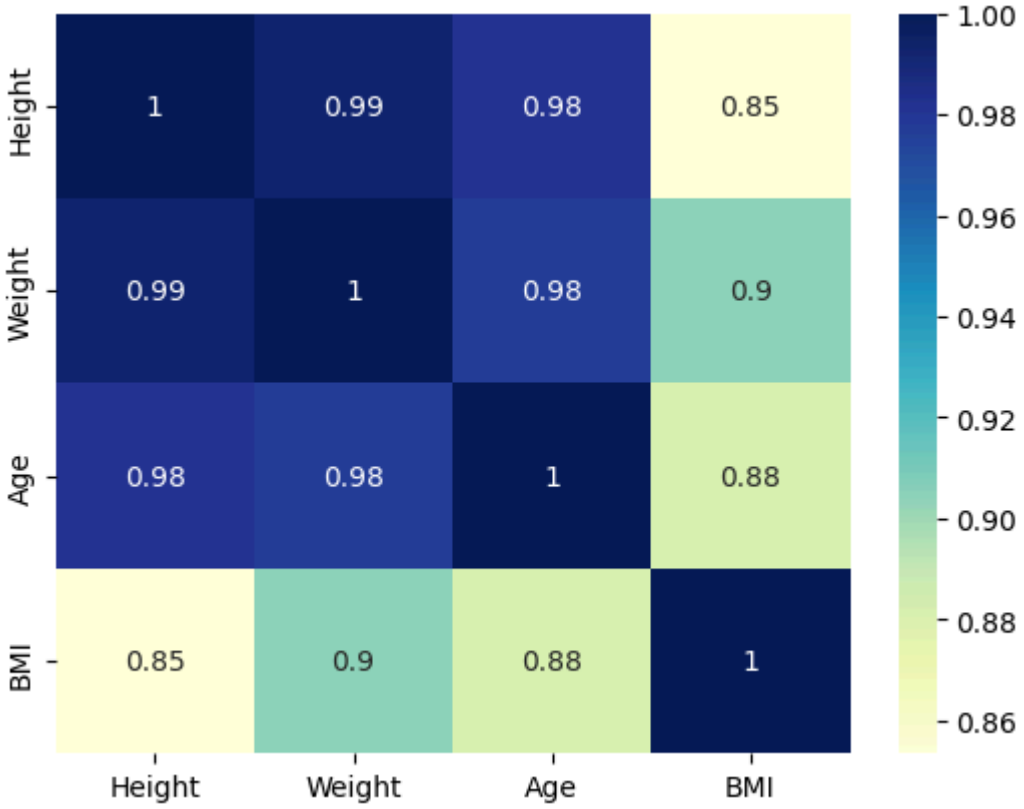
In [174…
```
# 151


df = pd.DataFrame({
    "Math":[78,85,90,66,88],
    "Science":[80,82,89,70,85],
    "English":[75,88,92,60,84],
    "Computer":[90,91,95,72,89]
```

```
}, index=["S1","S2","S3","S4","S5"])

sns.heatmap(df, annot=True, cmap="coolwarm", linewidths=1, linecolor="black")
plt.show()
```



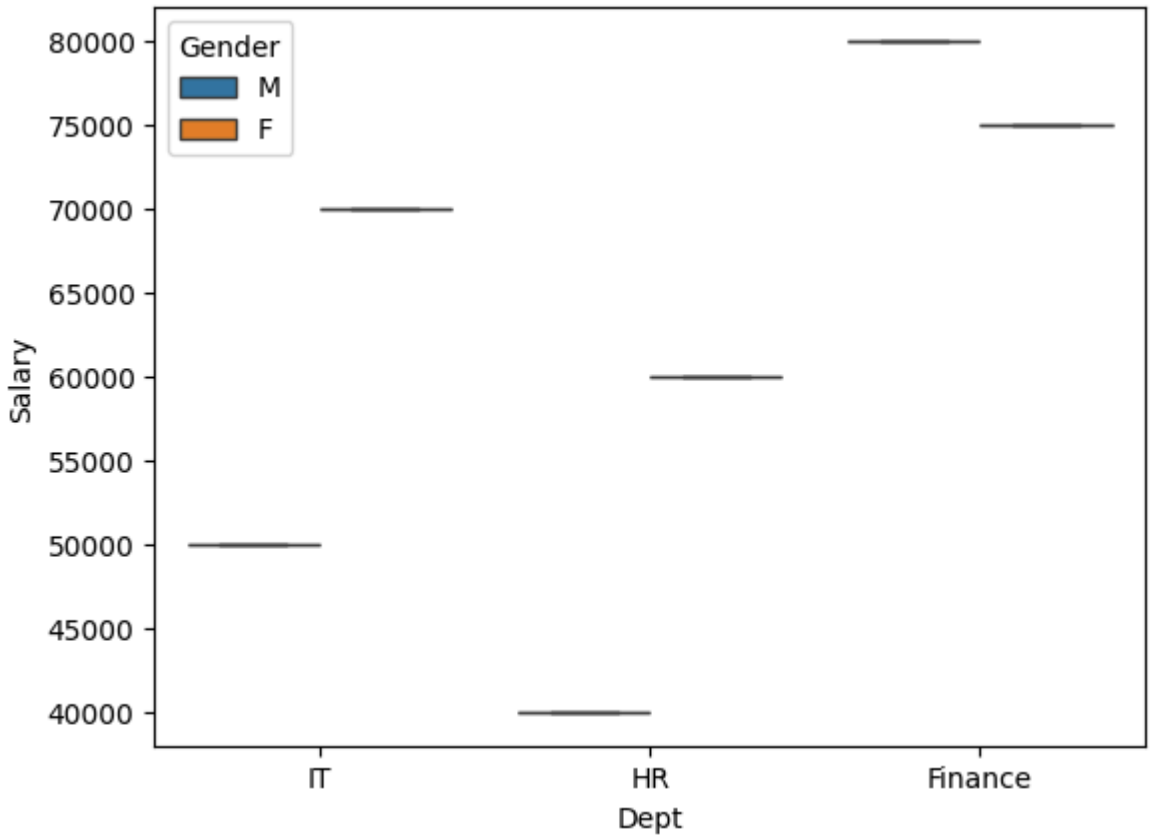
In [175…

```
# 152

df = pd.DataFrame({
    "Height":[150,160,170,180,175],
    "Weight":[50,60,65,75,70],
    "Age":[20,22,25,30,28],
    "BMI":[22,24,23,26,25]
})

sns.heatmap(df.corr(),cmap="YlGnBu", annot=True )
plt.show()
```



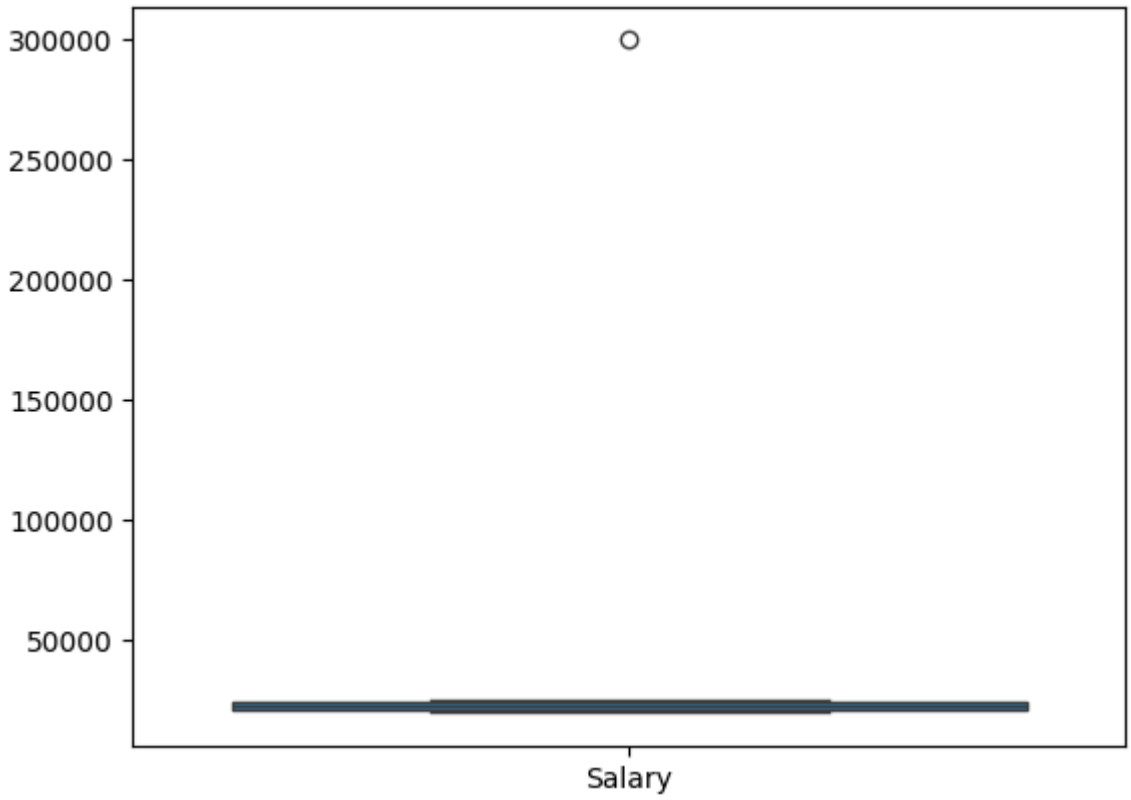
In [176…

```
# 153

df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Salary":[50000,70000,40000,60000,80000,75000],
    "Gender":["M","F","M","F","M","F"]
})

sns.boxplot(x="Dept", y="Salary", hue="Gender", data=df)
plt.show()
```



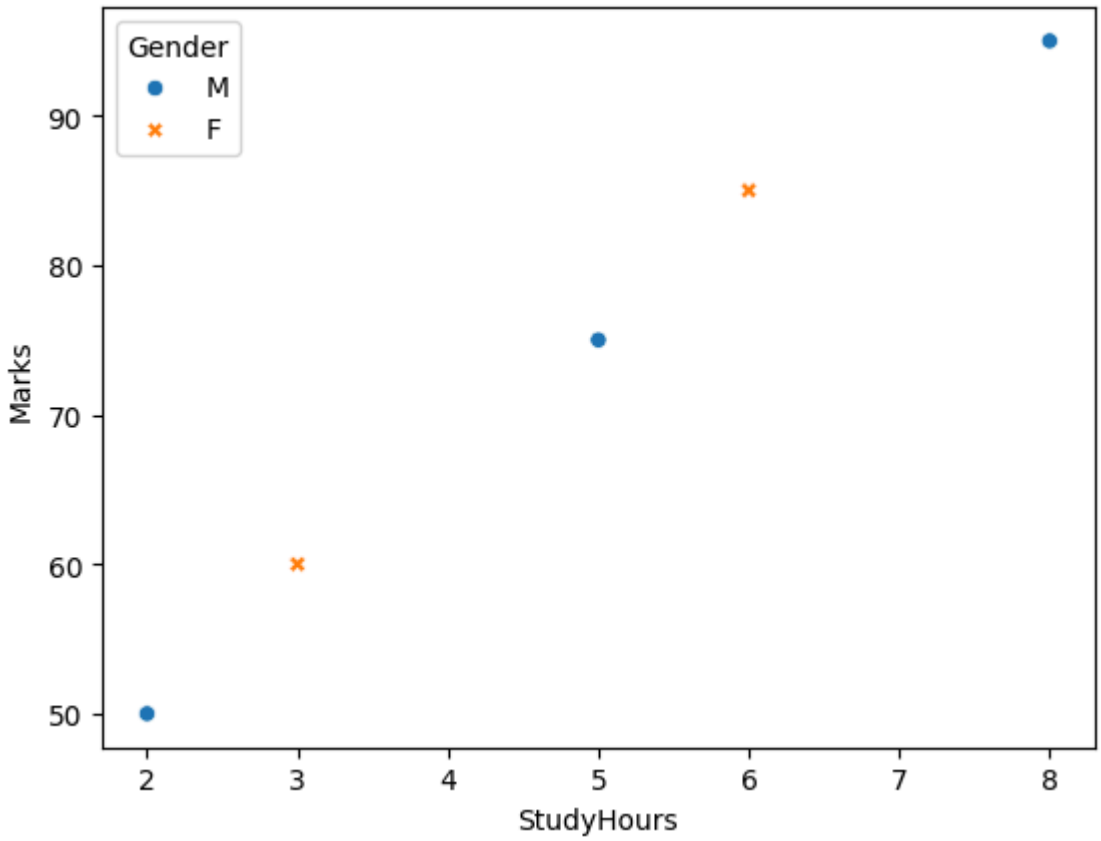
In [177…

```
# 154

df = pd.DataFrame({
    "Salary":[20000,22000,21000,23000,25000,300000]
})
sns.boxplot(df)
plt.show()
```



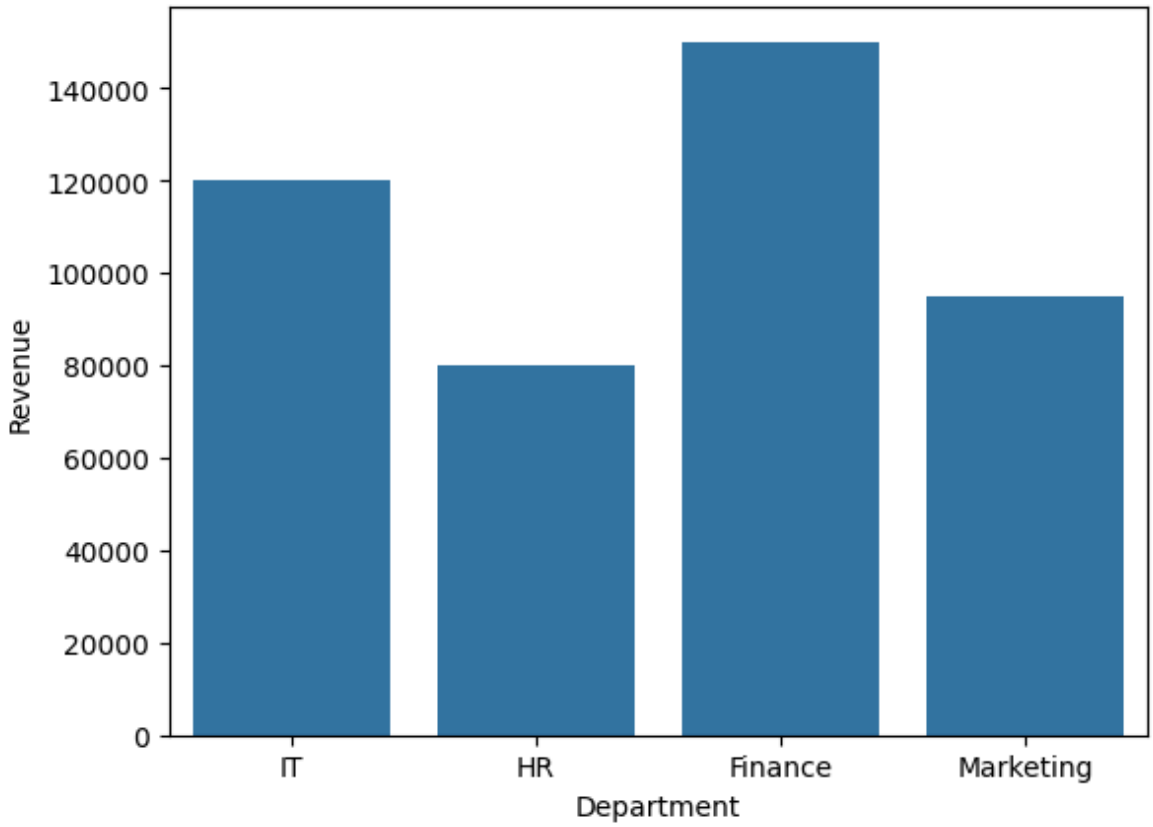
In [178…

```
# 155

df = pd.DataFrame({
    "StudyHours":[2,3,5,6,8],
    "Marks":[50,60,75,85,95],
    "Gender":["M","F","M","F","M"]
})

sns.scatterplot(x="StudyHours", y="Marks", hue="Gender", data=df,style="Gender")
plt.show()
```



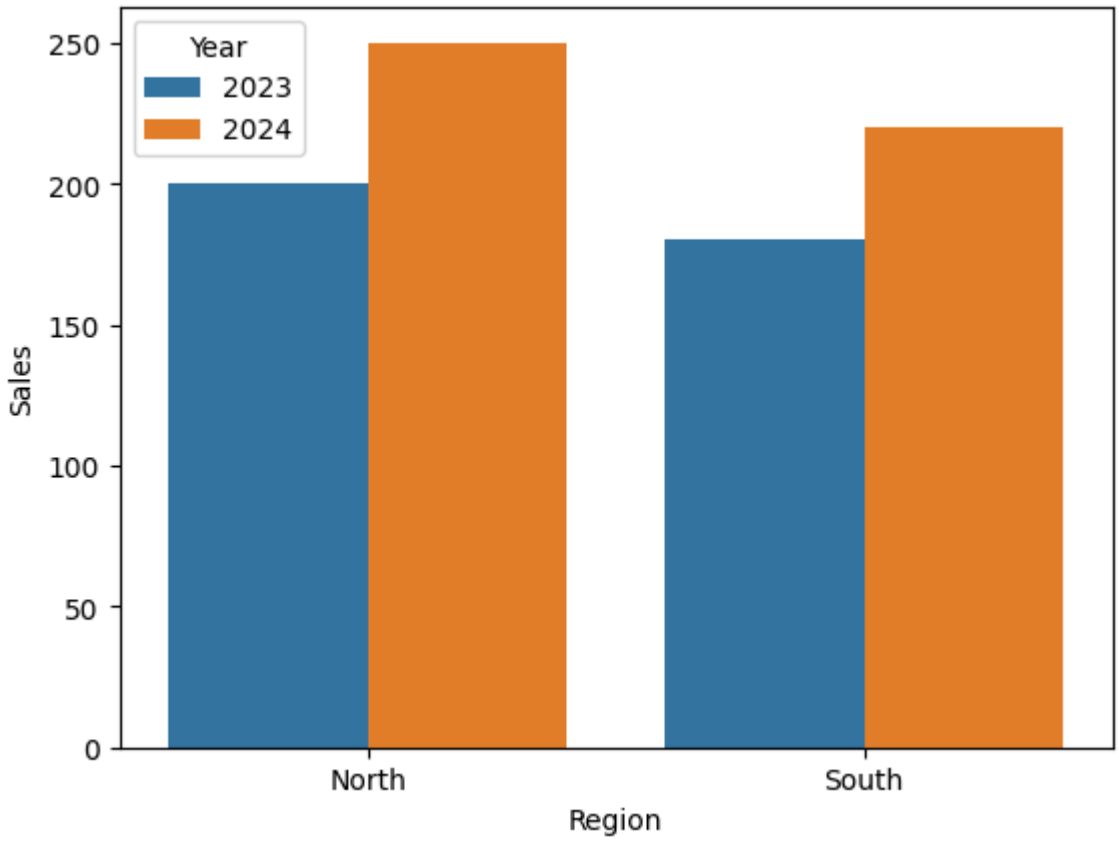
In [179…

```
# 156

import plotly.express as px
import plotly.io as pio
pio.renderers.default = "notebook"

df = pd.DataFrame({
    "Department": ["IT", "HR", "Finance", "Marketing"],
    "Revenue": [120000, 80000, 150000, 95000]
})

sns.barplot(x="Department", y="Revenue", data=df)
plt.show()
```



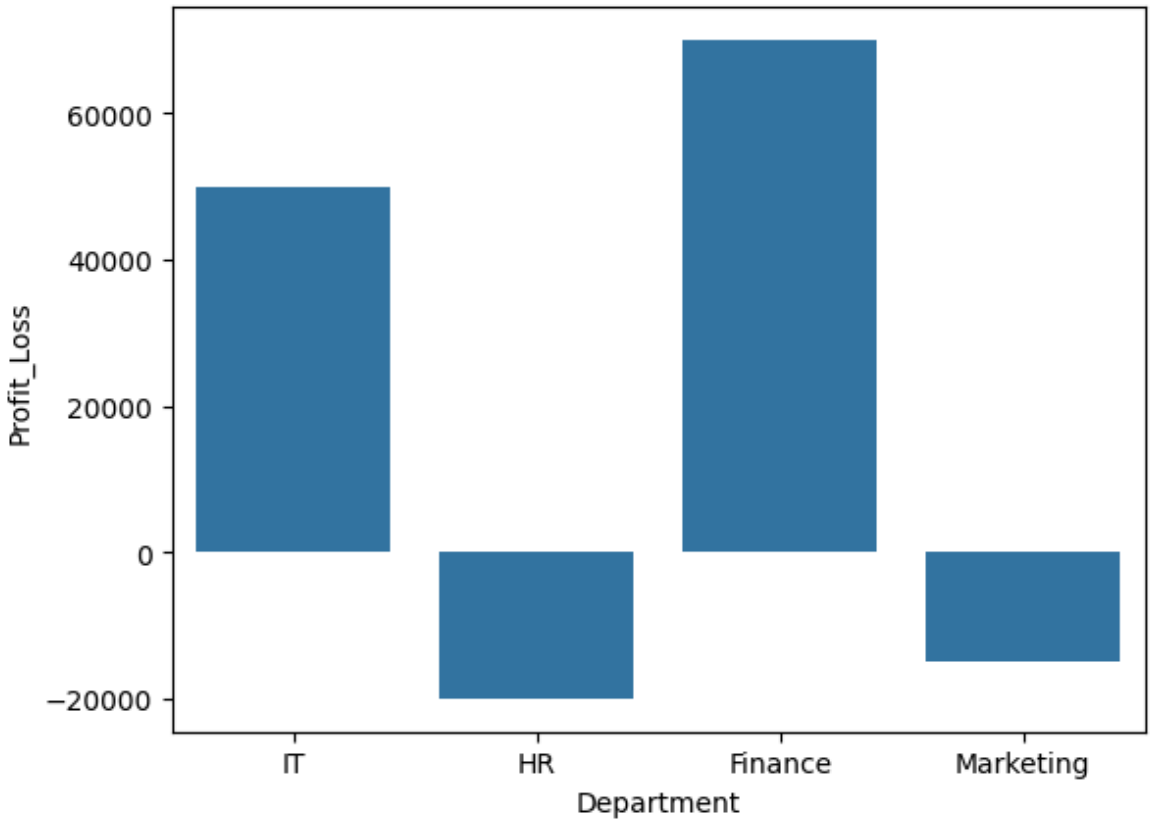
In [180…

```
# 157

import plotly.express as px
import plotly.io as pio
pio.renderers.default = "notebook"

df = pd.DataFrame({
    "Region": ["North","North","South","South"],
    "Year": ["2023","2024","2023","2024"],
    "Sales": [200,250,180,220]
})
sns.barplot(x="Region", y="Sales", hue="Year", data=df)
plt.show()
```



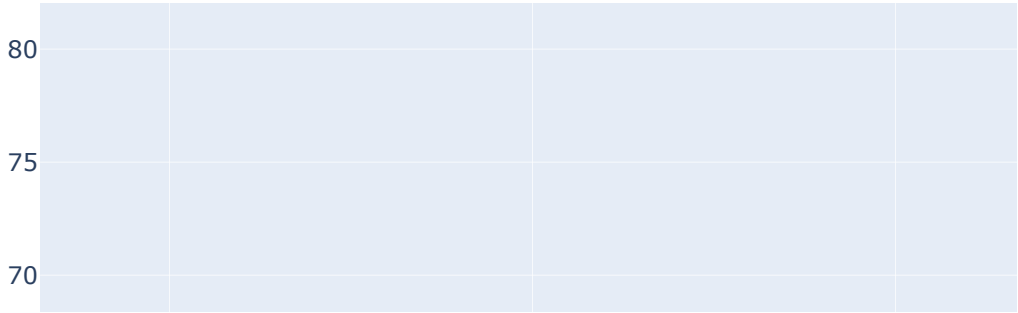
In [181…

```
# 158


pio.renderers.default = "notebook"

df = pd.DataFrame({
    "Department": ["IT","HR","Finance","Marketing"],
    "Profit_Loss": [50000,-20000,70000,-15000]
})

sns.barplot(x="Department", y="Profit_Loss", data=df)
plt.show()
```



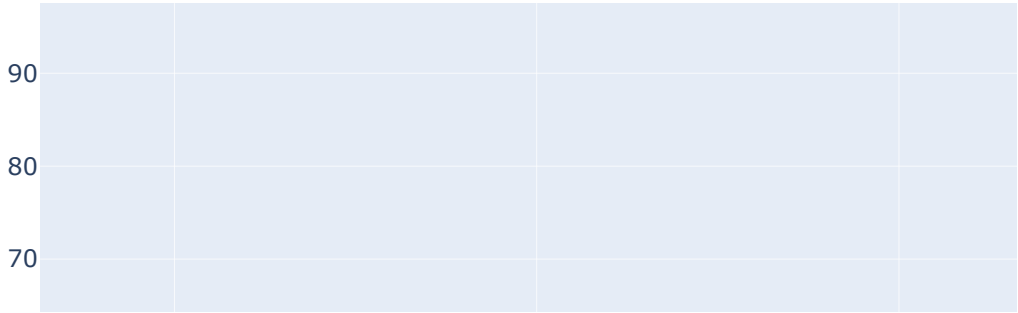
In [182…

```
# 159

pio.renderers.default = "notebook"
df = pd.DataFrame({
    "Height":[150,160,170,180,165],
    "Weight":[50,60,70,80,65],
    "Gender":["M","F","M","F","M"]
})

fig = px.scatter(df, x="Height", y="Weight", color="Gender")
fig.show()
```



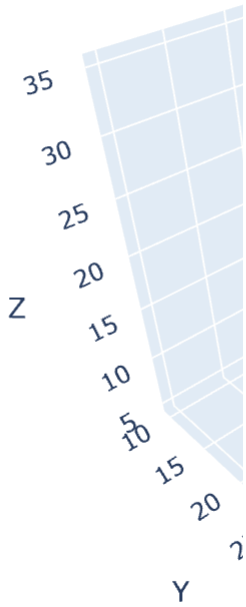
In [183…

```
# 160

df = pd.DataFrame({
    "Sales":[100,200,300,400],
    "Profit":[20,50,70,90],
    "Quantity":[10,30,50,80]
})

fig = px.scatter(df, x="Sales", y="Profit",size="Quantity")
fig.show()
```



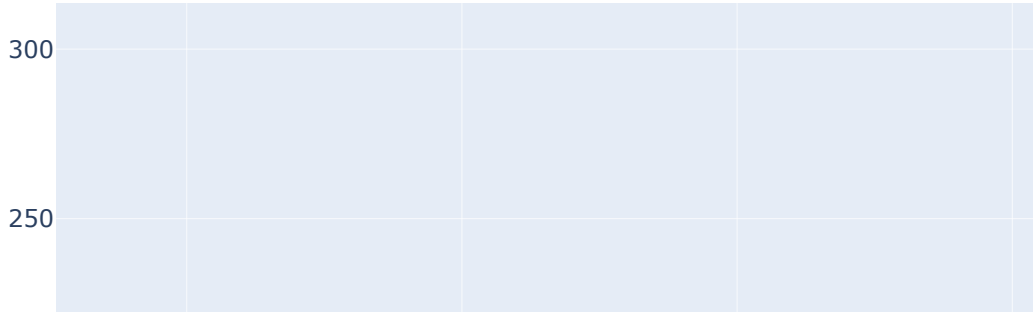
In [184…

```
# 161

df = pd.DataFrame({
    "X":[1,2,3,4],
    "Y":[10,20,30,40],
    "Z":[5,15,25,35],
    "Category":["A","B","A","B"]
})

fig = px.scatter_3d(df, x="X", y="Y", z="Z", color="Category")
fig.show()
```



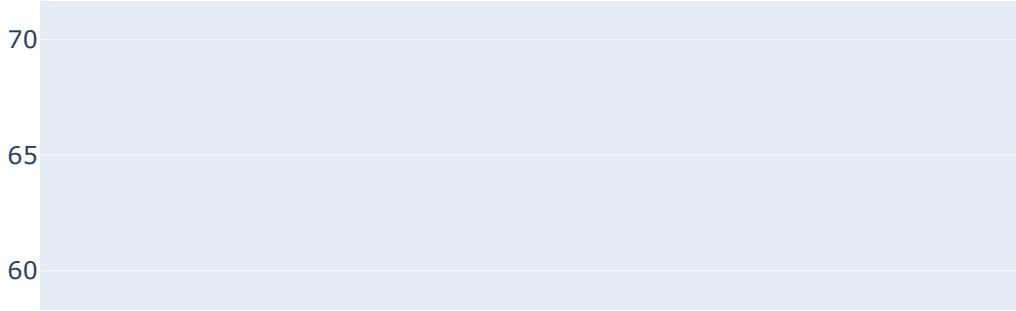
In [185…

```
# 162

df = pd.DataFrame({
    "Year":[2020,2021,2022,2023,2024],
    "Revenue":[100,150,200,250,300]
})

fig = px.line(df, x="Year", y="Revenue", markers=True)
fig.show()
```



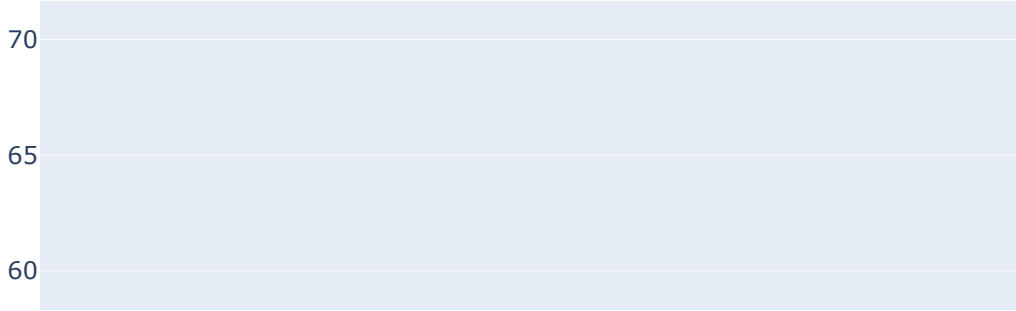
In [186…

```
# 163

df = pd.DataFrame({
    "Month":["Jan","Feb","Mar","Jan","Feb","Mar"],
    "Sales":[50,60,70,40,55,65],
    "Product":["A","A","A","B","B","B"]
})
fig = px.line(df, x="Month", y="Sales", color="Product")
fig.show()
```



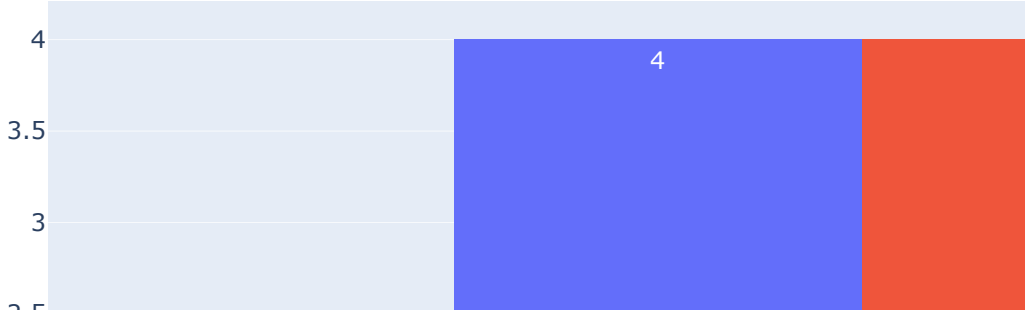
In [187…

```
# 164

df = pd.DataFrame({
    "Month":["Jan","Feb","Mar","Jan","Feb","Mar"],
    "Sales":[50,60,70,40,55,65],
    "Product":["A","A","A","B","B","B"]
})

fig = px.line(df, x="Month", y="Sales", color="Product")
fig.show()
```



In [188…

```
# 165
df = pd.DataFrame({
    "Student": [f"S{i}" for i in range(1, 21)],
    "Marks": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90,
              48, 52, 58, 62, 68, 72, 78, 82, 88, 92],
    "Grade": ["C","C","C","B","B","B","A","A","A","A",
              "C","C","C","B","B","B","A","A","A","A"]
})

fig = px.histogram(df,x = "Marks", color = "Grade", text_auto = True,nbins = 5)
fig.show()
```

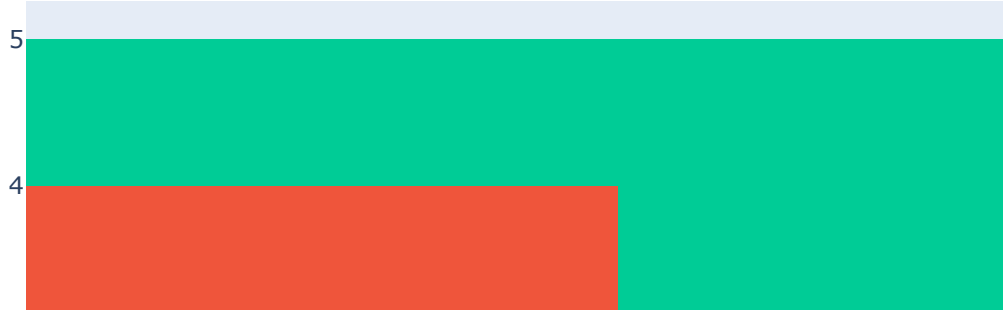


In [189…

```
# 166

df = pd.DataFrame({
    "Sales":[100,120,150,180,200,220,250,270,300,320,
             110,130,160,190,210,230,260,280,310,330,
             105,125,155,185,205,225,255,275,305,325],
    "Region":["North"]*15 + ["South"]*15
})

fig = px.histogram(df,x="Sales",color="Region",pattern_shape="Region",nbins=6)
fig.show()
```

In [190…

```
# 167


df = pd.DataFrame({
    "Age":[22,25,28,30,35,40,45,50,29,33,
           26,31,38,42,48],
    "Department":["IT","IT","HR","HR","Finance","Finance",
                  "IT","HR","Finance","IT",
                  "HR","Finance","IT","HR","Finance"]
})

fig = px.histogram(df,x="Age",color="Department",histfunc="count",nbins=5)
fig.show()
```



In [191…

```
# 168

df = pd.DataFrame({
    "Product":["A","B","C","D","E"],
    "Sales":[30,25,20,15,10]
})

fig = px.pie(df, names="Product", values="Sales", title="Salary Distribution by
fig.update_traces(textinfo= 'percent+label', textposition = 'inside')

fig.show()
```

Salary Distribution by Department

In [192…

```
# 169

df = pd.DataFrame({
    "Department":["IT","HR","Finance","Marketing"],
    "Budget":[50000,30000,40000,20000]
})

fig = px.pie(df,names="Department", values="Budget" ,title="Hacker bolti public"
fig.update_traces(textposition = "inside",textinfo = "percent+label" ,pull=[0.1,
fig.show()
```

Hacker bolti public

In [193…

```
# 170

df = pd.DataFrame({
    "City":["City A","City B","City C","City D"],
    "Population":[400,300,200,100]
})

fig = px.pie(df,names="City", values="Population" ,title="Hacker bolti public",
fig.update_traces(textposition = "inside",textinfo = "percent+label" ,pull=[0,0.
fig.show()
```

Hacker bolti public